

# normet

Normalisation, Decomposition and Counterfactual Modelling for Environmental Time Series

**User Manual**

Version 1.0.0

A desktop application and Python library for removing the weather from air-quality records and asking what an intervention changed.

October 24, 2026

# Contents


**1 Introduction** **4**
- 1.1 What is normet? . . . 4
- 1.2 The idea in one equation . . . 4
- 1.3 Key features . . . 5
- 1.4 How this manual is organised . . . 5

**2 Installation** **6**
- 2.1 macOS . . . 6
- 2.2 Windows and Linux . . . 6
- 2.3 From PyPI . . . 6

**3 Methodology** **8**
- 3.1 Preparing the data . . . 8
  - 3.1.1 The split . . . 8
- 3.2 Training the model . . . 9
  - 3.2.1 Model statistics . . . 9
- 3.3 Meteorological normalisation . . . 9
- 3.4 Emission decomposition . . . 10
- 3.5 Meteorological decomposition . . . 10
- 3.6 Rolling normalisation . . . 11
- 3.7 Multi-scale decomposition . . . 11
- 3.8 Partial dependence . . . 11
- 3.9 Zero-shot de-weathering with Chronos-2 . . . 12
- 3.10 Synthetic control . . . 12
  - 3.10.1 Estimators . . . 12
  - 3.10.2 Diagnostics . . . 13
  - 3.10.3 Inference . . . 13
  - 3.10.4 Staggered adoption . . . 14

**4 Getting started** **15**
- 4.1 The window before a table is open . . . 15
- 4.2 The examples . . . 16
- 4.3 A first analysis . . . 16

**5 The interface** **17**
- 5.1 The parameter panel . . . 17
- 5.2 Result tabs . . . 17
- 5.3 The action bar . . . 18
- 5.4 Run buttons . . . 19
- 5.5 Menus and shortcuts . . . 20
- 5.6 The log and the Assistant . . . 20

5.7 Appearance and language . . . 20

**6 Training a model** **22**
6.1 The table and its columns . . . 22
6.2 Settings . . . 22
6.3 Reading the Model tab . . . 24
6.4 Chronos-2 . . . 25

**7 Normalising** **26**

**8 Decomposing** **28**
8.1 By time scale: the emission decomposition . . . 28
8.2 By variable: the meteorological decomposition . . . 29
8.3 Trend . . . 29
8.4 Uncertainty decomposition . . . 31

**9 Rolling and multi-scale** **33**
9.1 Rolling normalisation . . . 33
9.2 Multi-scale decomposition . . . 33

**10 Partial dependence** **35**

**11 Synthetic control (SCM Studio)** **36**
11.1 Panel data and columns . . . 36
11.2 Design . . . 36
11.3 Fitting . . . 37
11.4 Inference . . . 38

**12 Getting data in** **41**
12.1 Input format . . . 41
12.2 The clock: time zone . . . 41
12.3 Data Studio: air quality networks and meteorology . . . 42
12.4 Transport Studio: back-trajectory predictors . . . 43
12.5 Polar plot: where a pollutant comes from . . . 44

**13 Exporting and reproducing** **46**
13.1 Tables and reports . . . 46
13.2 The command log . . . 46
13.3 Figures for publication . . . 47
13.4 Saving and reopening a session . . . 48
13.5 A Methods paragraph . . . 50
13.6 Configurations and history . . . 50

**14 The Assistant** **51**
14.1 Full automatic . . . 51
14.2 Choosing a model . . . 51

**15 Scripting: API, CLI and MCP** **53**
15.1 Python API . . . 53
15.2 Command line . . . 54
15.3 AI assistants over MCP . . . 55

**A Defaults** **56**

**B Where normet keeps its files** **57**

**C Troubleshooting** **58**

# Chapter 1

# Introduction

## 1.1 What is normet?

normet is a desktop application and Python library for the analysis of environmental time series, air-quality records above all. It answers two questions that a raw concentration record cannot answer by itself:

1. *How much of the change I see is the weather?* A month of stagnant, cold anticyclonic air raises concentrations as surely as a new source does. normet learns how the pollutant depends on the meteorology and the calendar, then averages the meteorology out, leaving the part of the record the weather does not explain. This is *meteorological normalisation*, or *de-weathering*.
2. *What would have happened without the intervention?* A low-emission zone, a lockdown or a plant closure changes one place at one date. normet builds a counterfactual for that place from other places the intervention did not touch — a *synthetic control* — and measures the difference after the date, with placebo tests and uncertainty bands to say whether the difference is larger than chance.

Everything the window does is a call into the normet Python package, which can be scripted, run from the command line, or driven by an AI assistant over the Model Context Protocol (Chapter 15). The window writes down each call it makes, so an analysis done by clicking can be saved as a script that repeats it (Section 13.2).

## 1.2 The idea in one equation

Let $y_t$ be the concentration at time $t$, $\mathbf{m}_t$ the meteorology at that time (wind speed and direction, temperature, humidity, boundary-layer height, ...) and $\mathbf{c}_t$ the calendar (a trend counter, the day of the year, the weekday, the hour). normet fits a machine-learning model

$$y_t \approx f(\mathbf{m}_t, \mathbf{c}_t) \tag{1.1}$$

and then asks what $f$ predicts at time $t$ if the weather at $t$ had been any other weather the site has seen. Averaging over many such draws,

$$\tilde{y}_t \;=\; \frac{1}{N}\sum_{k=1}^{N} f(\mathbf{m}^{(k)}, \mathbf{c}_t), \qquad \mathbf{m}^{(k)} \sim \text{the record's own meteorology}, \tag{1.2}$$

gives the *normalised* series $\tilde{y}_t$: the concentration expected at time $t$ under typical weather. Because the calendar $\mathbf{c}_t$ is kept, a change in emissions — which the model can only express through the trend, season, week and hour — survives in $\tilde{y}_t$, while the day-to-day swings that the weather caused are averaged away. Chapter 3 develops this and the decompositions built on it.

## 1.3 Key features

- AutoML training with FLAML (an estimator search within a time budget) or a tuned LightGBM, with the held-out fit reported as $R^2$, RMSE and nine further statistics.
- Monte-Carlo meteorological normalisation, with a choice of which variables are resampled, uncertainty quantiles, reference pools and conditional resampling; an adaptive version that stops when the series has converged.
- Emission decomposition (trend, seasonal, weekly, diurnal, short-term) and meteorological decomposition (one contribution per variable or named group, sequential or Shapley).
- Rolling (windowed) normalisation, and a multi-scale decomposition that separates the weather's influence by timescale.
- Partial-dependence curves of the trained model.
- Zero-shot de-weathering with the Chronos-2 time-series foundation model, on the same result schema — no training needed.
- Synthetic control on panel data: augmented SCM, classic Abadie SCM, difference-in-differences, matrix completion, robust SCM, ML-SCM and Bayesian SCM, with placebo-in-space and placebo-in-time tests, jackknife and bootstrap bands, and an all-units batch.
- Data Studio: air quality from the six UK networks, the EEA reporting network for Europe, or OpenAQ for most of the world, with reanalysis meteorology, merged into a model-ready table (no key for the UK networks or the EEA).
- Transport Studio: HYSPLIT back-trajectories turned into transport predictors and joined onto the record.
- A command log that writes every action as the call that repeats it; HTML reports with provenance; figures laid out for a journal's column.
- An Assistant (a language model working in the window) that proposes the target and predictors and reads the results with you.

## 1.4 How this manual is organised

Chapter 2 covers installation. Chapter 3 explains the methods in enough depth to interpret every number the program reports. Chapters 4–10 walk through a complete de-weathering analysis in the window on a real record, and Chapter 11 the synthetic-control workflow on a real panel. Chapter 12 covers getting data in, Chapter 13 exporting, reporting and reproducing a session, Chapter 14 the Assistant and Chapter 15 scripting. The appendices list the defaults, where normet keeps its files, and a troubleshooting guide.

The figures are screenshots of the real window, made by `docs/manual/make_figures.py` on the record normet ships with: hourly $NO_2$ and ERA5 meteorology at London Marylebone Road (site MY1), January to August 2020, a window that spans the UK's first COVID-19 lockdown. The numbers quoted in the text are the ones that script printed. A run of your own will differ in the last digit or two: the AutoML search is bounded by wall-clock time, so it does not visit exactly the same candidates twice.

This manual is also published in Chinese (`normet_manual_zh.pdf`), chapter for chapter, with its screenshots of the window in Chinese (Section 5.7).

# Chapter 2

# Installation

normet ships as a self-contained desktop application for each platform, and as a Python package on PyPI. Choose the installers for interactive analysis, the package for scripting, servers and notebooks.

## 2.1 macOS

Open `Normet-1.0.0-macos-arm64.dmg` and drag `Normet.app` onto the `Applications` folder. The application carries its own Python, Qt, FLAML, LightGBM and XGBoost, and the Data Studio's readers, so nothing else is needed for the de-weathering, synthetic-control and data workflows. The first launch takes longer than the rest while it builds a font cache, which later launches reuse (it is kept in `~/.normet/matplotlib`); a splash screen shows meanwhile.

**Note.** The application is not signed with an Apple Developer ID. The first time, open it by right-clicking `Normet.app` and choosing *Open*, then confirming in the dialog; this is needed only once. Alternatively run `xattr -dr com.apple.quarantine /Applications/Normet.app` in Terminal.

The installers do not include the optional Chronos-2 foundation model (it needs PyTorch, about a gigabyte on its own), the Bayesian SCM backend (PyMC) or the MCP server. Their options are greyed out in the window, with a tooltip naming what is missing; install normet from PyPI with the extra to use them (below).

## 2.2 Windows and Linux

On Windows run `normet-setup-1.0.0.exe`, an Inno Setup wizard that adds Start-menu and optional desktop shortcuts. The installer is unsigned, so SmartScreen shows *Windows protected your PC* on first run: choose *More info* → *Run anyway*. On Linux download `normet-1.0.0-x86_64.AppImage`, mark it executable (`chmod +x`) and run it. On a minimal distribution Qt may need `libegl1 libgl1 libxkbcommon0 libdbus-1-3 libxcb-cursor0`.

The installers are built by the repository's `build-gui.yml` workflow (one per operating system; PyInstaller cannot cross-compile) and are attached to its runs under *Actions* → *Artifacts*. `packaging/README.md` explains how to build one locally.

## 2.3 From PyPI

normet needs Python 3.10 or newer (3.10–3.12 are tested) and NumPy below 2. The core package is small; everything heavier is an extra:

```
python -m venv .venv && source .venv/bin/activate
pip install "normet[gui,flaml]" # the desktop app with the AutoML backend
normet-gui # or: normet-gui mydata.csv
```

| Extra | What it adds |
|---|---|
| `flaml` | FLAML AutoML backend (with XGBoost, which FLAML imports) — recommended |
| `lgb` | LightGBM backend with random hyperparameter search |
| `gui` | PySide6, for `normet-gui` |
| `data` | ERA5 (CDS), UK networks, OpenAQ, EEA, Open-Meteo adapters |
| `geo` | Shapely and pyshp: source regions from GeoJSON/Shapefile |
| `cli` | Click and PyYAML, for the `normet` command |
| `foundation` | Chronos-2 and PyTorch: zero-shot de-weathering, station embeddings |
| `finetune` | `foundation` plus `accelerate` and `peft`, to fine-tune Chronos-2 |
| `bayes` | PyMC and ArviZ, for the Bayesian SCM |
| `mcp` | The MCP server `normet-mcp`, for AI assistants |
| `dask` | Dask, for large data sets |
| `all` | Everything above |

Importing normet never requires an optional dependency: a function that needs one says which extra to install when it is called.

# Chapter 3

# Methodology

This chapter explains what normet computes, so that the numbers it reports can be interpreted with confidence. You do not need it to run an analysis; you do need it to defend one.

## 3.1 Preparing the data

Before a model is trained the table goes through `prepare_data`, in this order:

1. **A date column.** The table must have one datetime column called `date` (or an index, or a column named `datetime`, `time` or `timestamp`, which is renamed). Dates with time zones are converted to UTC and then made naive.
2. **Target and predictors.** The target is renamed `value`; the table is reduced to the date, the target and the predictors.
3. **Missing values.** Rows with a missing target are dropped. A missing numeric predictor is filled with that column's median; a missing categorical one with its most frequent value.
4. **Calendar variables.** Four columns are added: `date_unix` (seconds since 1970, the trend), `day_julian` (day of the year, the season), `weekday` (1 = Monday to 7 = Sunday, categorical) and `hour`. The model uses only those listed among its predictors; the window always lists all four.
5. **Train/test split.** A column `set` marks each row `training` or `testing` (Section 3.1.1).

### 3.1.1 The split

The held-out rows are what the model's fit is judged on. Four splits are offered:

`random` (default) A random 25 % of rows are held out. Every season, weekday and hour is represented in training.

`ts` The last 25 % of the record is held out. A trend variable must then be extrapolated past anything seen in training, which tree models cannot do; use it to test forecasting, not to train for normalisation.

`month_ts` Within every calendar month of every year, one contiguous block of 25 % is held out, at a random (seeded) position in the month.

`season_ts` The same within every meteorological season (DJF, MAM, JJA, SON; December counts with the following year's winter).

**Caution.** Under `random`, neighbouring hours fall on both sides of the split, so an autocorrelated record looks easier to predict than it is: the test $R^2$ is an upper bound. This matters most for predictors that are constant over several hours, such as back-trajectory features released every six hours, which a tree model can use as a time stamp. On the bundled MY1 $PM_{2.5}$ case a trajectory-only model scores about 0.84 under a random split and 0.46 under a blocked one. Judge such models with `month_ts`.

## 3.2 Training the model

normet trains gradient-boosted trees (by default) that map the predictors to the target. Two backends are offered:

**FLAML** An AutoML search over the estimators ticked (LightGBM alone by default; XGBoost, depth-limited XGBoost, random forest, extra trees and scikit-learn's histogram gradient boosting can be added) within a wall-clock *time budget*, maximising $R^2$. The library's default budget is 90 s; the window's is 60 s.

**LightGBM** LightGBM with a random hyperparameter search of a set number of *trials* (20 in the window).

A longer budget rarely hurts and often helps on long records. Because the FLAML budget is wall-clock time, the same script on the same data can stop at a different candidate on a busier machine; set `max_iter` in `model_config` with `time_budget=-1` for a search bounded by trials instead.

### 3.2.1 Model statistics

`modStats` predicts the training rows, the testing rows and all rows, and reports for each: $n$; FAC2 (the fraction of predictions within a factor of two of the observation); MB and MGE (mean bias and mean gross error); NMB and NMGE (the same, normalised by the sum of observations); RMSE; $r$ and its significance; COE (coefficient of efficiency); IOA (index of agreement); and $R^2$. The held-out (set = testing) row is the one to judge the model by.

## 3.3 Meteorological normalisation

Equation (1.2) is computed by Monte Carlo. For each of $N$ draws (`n_samples`, 300 by default):

1. The *resampled variables* of every row are replaced, all together, by those of a row drawn at random (with replacement) from the *resample pool* — by default the record itself. Whole rows are drawn, so the resampled variables keep their joint structure: a draw never pairs a winter temperature with a summer boundary layer.
2. Every other predictor — by default the four calendar variables — keeps its observed value.
3. The model predicts every row.

The normalised series is the mean of the $N$ predictions at each timestamp; with `return_quantiles` the chosen quantiles of the $N$ predictions are returned as well, as columns `q050`, `q950` and so on (the quantile $\times 1000$). These quantiles describe the spread of the prediction over weather, not the uncertainty of the model.

**Which variables to resample.** Resampling the meteorology and keeping the calendar is the usual de-weathering. Resampling a calendar variable too removes what it carries: resampling `day_julian` removes the seasonal cycle, resampling `date_unix` removes the trend. A predictor that is not meteorological — traffic counts, say — can be kept at its observed values by leaving it unticked.

**How many samples.** The Monte-Carlo error of each timestamp shrinks as $1/\sqrt{N}$. 300 is ample for a smooth daily or monthly mean; individual hours are noisier. `normalise_auto` chooses $N$ for you: it adds batches of 100 draws until, for three checks in a row, 95 % of the timestamps have a Monte-Carlo relative standard error below 3 %, or 5000 draws have been made (a warning says when the cap was hit).

**Reference pools.** `resample_df` draws the weather from another table — 2019's weather, for example, to ask what 2020 would have looked like under 2019's meteorology. `conditional_on` restricts the pool to rows meeting a condition — a value, a list of values, or a function of the column, as `{"ws": lambda v: v < 2}` draws from calm hours only. `resample_pools` draws some variables from a pool of their own: `{"transport": clean_hours[traj_cols]}` draws the trajectory features from clean-air-mass hours while the local weather is still drawn from the whole record. Each pool has its own random stream, so adding or renaming a pool does not move the draws of the others.

**Reproducibility.** The draws are seeded (`seed=7654321` by default), so the same model, data and settings give the same series to the last digit.

## 3.4 Emission decomposition

The emission decomposition splits the normalised series into the timescales the calendar variables carry, by *nested* normalisation. Five normalisations are run with the same seed, fixing one more calendar variable each time, in this fixed order:

| Run | Held at observed values | Resampled | Gives |
|---|---|---|---|
| $b$ | nothing | everything | base |
| $u$ | `date_unix` | the rest | trend $= u - b$ |
| $j$ | ... + `day_julian` | the rest | seasonal $= j - u$ |
| $w$ | ... + `weekday` | the rest | weekly $= w - j$ |
| $h$ | ... + `hour` | meteorology | diurnal $= h - w$ |

The last run, $h$, is the ordinary normalised series, reported as `emi_total`. The base run is split into its mean, `emi_base`, and its deviation from that mean, `emi_noise`; so

$$\text{emi_total} = \text{emi_base} + \text{date_unix} + \text{day_julian} + \text{weekday} + \text{hour} + \text{emi_noise}. \tag{3.1}$$

A calendar variable that is not among the model's predictors gets no column.

**Caution.** The order decides what each component can hold. The trend is computed while the day of the year, weekday and hour are still being averaged over, so it cannot carry a signal that recurs at the same date every year — only a genuine drift. The seasonal component is computed with each row's own trend value already fixed, so it is not a climatology: it stays specific to its year and can register a one-off event, such as a single year's holiday dip or a lockdown, despite its label. Look for a recurring calendar effect in `day_julian`, not `date_unix`.

## 3.5 Meteorological decomposition

The meteorological decomposition splits the part of the prediction the weather accounts for among the meteorological variables, or among named groups of them. The calendar is held at its observed values throughout. Write $v(S)$ for the normalised series with the variables in $S$ held at their observed values and the rest resampled; $v(\emptyset)$ is `emi_total`, and $v(\text{all})$ is the model's prediction with nothing resampled. The difference $v(\text{all}) - v(\emptyset)$ is split in one of two ways:

**Sequential** Fix the variables one at a time and report each step's change: $\phi_i = v(\{1, \ldots, i\}) - v(\{1, \ldots, i-1\})$. This takes $k+1$ normalisations for $k$ variables, but each contribution is conditional on those fixed before it, so the split depends on the order — an explicit `variable_order`, else the model's feature importance, which can reshuffle when the model is refitted.

**Shapley** Average each variable's marginal effect over every order it could be fixed in:

$$\phi_i = \sum_{S \not\ni i} \frac{|S|!\,(k - |S| - 1)!}{k!} \left[ v(S \cup \{i\}) - v(S) \right]. \tag{3.2}$$

No order is privileged. The exact values need all $2^k$ coalitions, allowed up to $k = 10$; beyond that, group the variables or pass `n_permutations` for an estimate from random orders (drawn in antithetic pairs).

Either way the contributions add up exactly to $v(\text{all}) - v(\emptyset)$, and every normalisation uses the same seed, so the differences between them are paired (common random numbers) rather than the difference of two noisy estimates. The result also carries `met_total` = observed − `emi_total`, its mean `met_base`, and `met_noise` = `met_total` − `met_base` $- \sum_i \phi_i$, the part of the observation the model does not reproduce.

**Groups.** `groups={"local": met_cols, "transport": traj_cols}` gives one column per group; a group is fixed and resampled as a unit, so its column includes the interactions among its members. Every meteorological predictor must be in exactly one group. With groups the default attribution is Shapley; without them it is sequential.

## 3.6 Rolling normalisation

`rolling` normalises the record window by window: windows of `window_days` days (14 by default), started every `rolling_every` days (7), each normalised with its weather drawn *from that window only*. A short window therefore removes only the weather that varies within it — synoptic variability for a fortnight — and keeps the slower meteorological variation that a full-record normalisation would remove. The result has one column per window, counted from 1: `window_01`, `window_02`, ... (padded to three digits past 99 windows, so the names sort in the windows' order); `window_columns(rolled)` lists them. The window shows their mean, with the spread across overlapping windows.

## 3.7 Multi-scale decomposition

Write $Y_W$ for the mean of the rolling-normalised series at window width $W$, and $Y_\infty$ for the ordinary normalised series, whose pool is the whole record. As $W$ grows the window's pool approaches the record's, and $Y_W \to Y_\infty$. Differences between widths isolate the weather influence specific to each band of timescales, much as wavelet detail coefficients do:

$$D_{\text{fast}} = Y_{\text{fast}} - Y_{\text{meso}} \qquad \text{synoptic, sub-monthly} \tag{3.3}$$

$$D_{\text{meso}} = Y_{\text{meso}} - Y_{\text{slow}} \qquad \text{intra-seasonal} \tag{3.4}$$

$$D_{\text{slow}} = Y_{\text{slow}} - Y_\infty \qquad \text{what even the slow window's pool leaves} \tag{3.5}$$

The widths default to 14, 90 and 365 days. Each scale's windows are stepped every quarter of the width, and a scale with fewer than two windows on the record is skipped, with a note saying so.

## 3.8 Partial dependence

A partial-dependence curve shows how the model's prediction responds to one predictor with the others averaged over the data: for each of `grid_points` values spanning the 1st to 99th percentile of the variable, every training row's value of it is set to that value and the predictions are

averaged. `pdp_mean` is that average and `pdp_std` the spread of the predictions about it. A curve describes the model, not the atmosphere; correlated predictors (temperature and boundary-layer height, say) are varied one at a time, into combinations the record may never contain.

## 3.9 Zero-shot de-weathering with Chronos-2

Chronos-2 is a pretrained time-series foundation model. normet uses it without training: the meteorology enters through the model's covariate channel, and the weather is marginalised by resampling exactly as in Equation (1.2). It is useful where there is too little history to train on, at a site that has just come online, or as a second estimate from a model that never saw the target's own record. Four properties decide whether it is the right tool:

- **The first `context_length` rows are not a result.** The model needs that much history (2048 rows by default) before it can project, so those rows are seeded with the observed values and the two curves coincide there by construction. Give it years, not months.
- **Every Monte-Carlo sample is a full forward pass**, so the default is 8 samples, not 300. The model runs on CUDA, Apple Silicon (`mps`) or the CPU, in that order of preference; the CPU is slow.
- **Position is time.** Chronos-2 counts rows, so the record must be on a regular grid; `to_regular_index` rebuilds it, leaving gaps as missing.
- **Covariate sensitivity is the quality gate.** Nothing was fitted, so there is no held-out fit and no feature importance. Instead the future weather is shuffled and the shift in the forecast's median measured. Below about 1 % of the prediction the model is reading the target's own history and ignoring the weather, and a "normalised" series from it means nothing.

The meteorological decomposition has a zero-shot path. The emission decomposition, the rolling normalisation and partial dependence do not: they are defined on a fitted model's response to the calendar variables, which Chronos-2 reads off the target's own history instead.

## 3.10 Synthetic control

A synthetic control estimates the effect of an intervention on one *treated* unit (a site, a city) from a panel of units observed before and after a *cutoff* date. Units the intervention did not touch — the *donor pool* — are combined into a synthetic version of the treated unit that tracks it before the cutoff; after the cutoff, the synthetic series is the counterfactual, and

$$\text{effect}_t = \text{observed}_t - \text{synthetic}_t. \tag{3.6}$$

The average effect over the post-cutoff period is reported as the ATT (average treatment effect on the treated). For an air-quality outcome, a de-weathered series is the natural input: the weather is then not part of what the donors must reproduce.

### 3.10.1 Estimators

**`scm`** Augmented SCM (the default). At each date a ridge regression across the donors predicts the outcome from each unit's pre-cutoff trajectory; the synthetic value is the treated unit's ridge prediction plus a weighted sum of the donors' ridge residuals, the weights summing to one (non-negative unless allowed otherwise).

**`abadie`** Classic SCM: non-negative donor weights summing to one, fitted to the treated unit's pre-cutoff path. Interpretable, but it cannot reach a treated unit that lies outside the range of its donors.

`did` Difference-in-differences: a two-way fixed-effects baseline that assumes parallel trends.

`mcnnm` Matrix completion with nuclear-norm minimisation (Athey et al., 2021): the post-cutoff treated cells are treated as missing and filled from a low-rank fit of the panel.

`robust` Robust SCM (Amjad, Shah and Shen, 2018): the donor matrix is de-noised by singular-value thresholding before the weights are fitted.

`mlscm` ML-augmented SCM (*experimental*): an AutoML model maps donor outcomes to the treated outcome in the pre-period. It lacks the causal guarantees of the others and warns so.

`bayesian` Bayesian simplex SCM (PyMC): a Dirichlet prior on the weights, giving posterior credible bands directly.

### 3.10.2 Diagnostics

`scm_diagnostics` reports the pre-cutoff fit (`pre_rmse`, `pre_mae`, `pre_mape`, `pre_r2`), the post-cutoff effect (`att`, `att_cum`, `post_rmse`) and how concentrated the weights are: the Herfindahl index $\mathrm{HHI} = \sum_j w_j^2$, the effective number of donors $1/\mathrm{HHI}$, and the top donors with their share.

**Caution.** A good pre-cutoff fit is evidence only if the estimator could not have fitted any series as well. Every estimator that weights donors to match the pre-period overfits when the donors are many for the pre-cutoff dates, and augmented SCM (`scm`) most of all: its ridge features are the units' pre-cutoff outcomes, so at a pre-cutoff date the treated unit's own value at that date is one of the features it is predicted from, and with as many donors as dates it reproduces almost any pre-period. On the ULEZ panel of Chapter 11 (27 pre-cutoff months), a treated series replaced by random noise is fitted with $R^2 = -0.05$ from 5 donors, 0.69 from 20 and 0.997 from 53; classic SCM, with 40 random-walk donors over 12 months, fits a shuffled series with 0.60. So the SCM Studio refits each design with the treated unit's pre-cutoff values shuffled in time, and reads the pre-cutoff $R^2$ only when that shuffled series cannot also reach 0.5 (Section 11.3). The `scm` estimator's donor weights balance only the ridge residuals, so their concentration does not describe which donors the synthetic unit is made of.

### 3.10.3 Inference

**Placebo in space** Each donor in turn is treated as if it had been treated, and given a synthetic control built from the *other* donors (the real treated unit is never a donor to a placebo). With the post-cutoff mean effect as the statistic, the permutation $p$-value is

$$p = \frac{1 + \#\{\, j : |\bar{e}_j| \geq |\bar{e}_{\text{treated}}| \,\}}{1 + J}, \tag{3.7}$$

for $J$ placebo units; the smallest it can be is $1/(J+1)$. The placebo effects also give a pointwise band for the treated effect.

**Placebo in time** The cutoff is moved to earlier dates (every `placebo_every` steps, keeping at least `min_pre_period` steps before it), where nothing happened. The true effect is ranked against the fake cutoffs' effects in the same way. Every fake post-period must end before the true cutoff.

**Jackknife bands** The fit is repeated leaving out each donor in turn; the spread of the effect gives a pointwise band. A result carried by one donor shows here.

**Bootstrap bands** The fit is repeated $B$ times on random subsets of the donors (80 % by default), optionally with the pre-cutoff period resampled in blocks.

**Conformal interval** `conformal_effect_interval` gives a finite-sample interval for the average post-period effect (Chernozhukov, Wüthrich and Zhu, 2021); `rmspe_ratio_test` gives Abadie's post/pre RMSPE-ratio test.

### 3.10.4 Staggered adoption

When the intervention rolls out unit by unit — the UK ULEZ 2021 expansion, staggered driving restrictions across cities — each adopting unit has its own cutoff. `run_scm_staggered` fits each treated unit on its own pre- and post-period, with the *other treated units excluded from every run's donor pool*, so one unit's intervention cannot leak into another unit's synthetic control. Each unit's effect is the post-cutoff average of its effect path,

$$\bar{e}_j = \frac{1}{|T_j|} \sum_{t \geq c_j} (\text{observed}_t - \text{synthetic}_t), \tag{3.8}$$

where $c_j$ is the unit's cutoff and $T_j$ its post-cutoff dates; the result is one row per unit ($c_j$, the pre- and post-period lengths, the pre-period fit $R^2$, the effect, and $p_j$, the unit's own placebo-in-space test at its own cutoff) plus mean and median rows over the units' effects.

The *mean* row's $p$-value is the cross-unit permutation test: each donor in turn takes every treated unit's role — the full set of cutoffs — and its pseudo-effects are averaged the same way the real units' are,

$$p = \frac{1 + \#\{\, u : |\bar{e}_u| \geq |\bar{\bar{e}}| \,\}}{1 + J}, \tag{3.9}$$

for $J$ donors and $\bar{\bar{e}}$ the mean of the treated units' effects. The real treated units are not permuted into the reference: their data already carry the intervention, and in a staggered design a unit given a later cutoff than its own has its real effect inside its new "pre-period", while one given an earlier cutoff has a "post-period" that is partly effect-free. Permuting treated identities is contaminated by the true effect, and with few treated units the test degenerates — with two there is one non-identity permutation, and it is contaminated. A donor-only reference is the clean null: a donor never intervened on draws from the null distribution. Each placebo fit needs a pool of its own, and with the default `scm` backend that is at least three donors; with a smaller pool the $p$-values come back NaN while the effects are still read.

# Chapter 4

# Getting started

## 4.1 The window before a table is open

normet opens on its **Data** tab with nothing loaded (Figure 4.1), the parameter panel folded away and the Assistant (Chapter 14) open beside the results. *Expand Panel* on the action bar, or Ctrl+B, brings the panel out; **View** → **Assistant** closes the Assistant. Under *Start with a time series* are two buttons — *Open Data…* and *Ask the Assistant* — and cards for the ways in: the examples, the five tables opened most recently (each with its folder, size and the date it last changed; one that has since moved is greyed out), and the two places data can come from without a file of your own, the Data Studio and the Transport Studio. A card with nothing to list is left out. Dropping a table file (Section 12.1) anywhere on the window opens it.

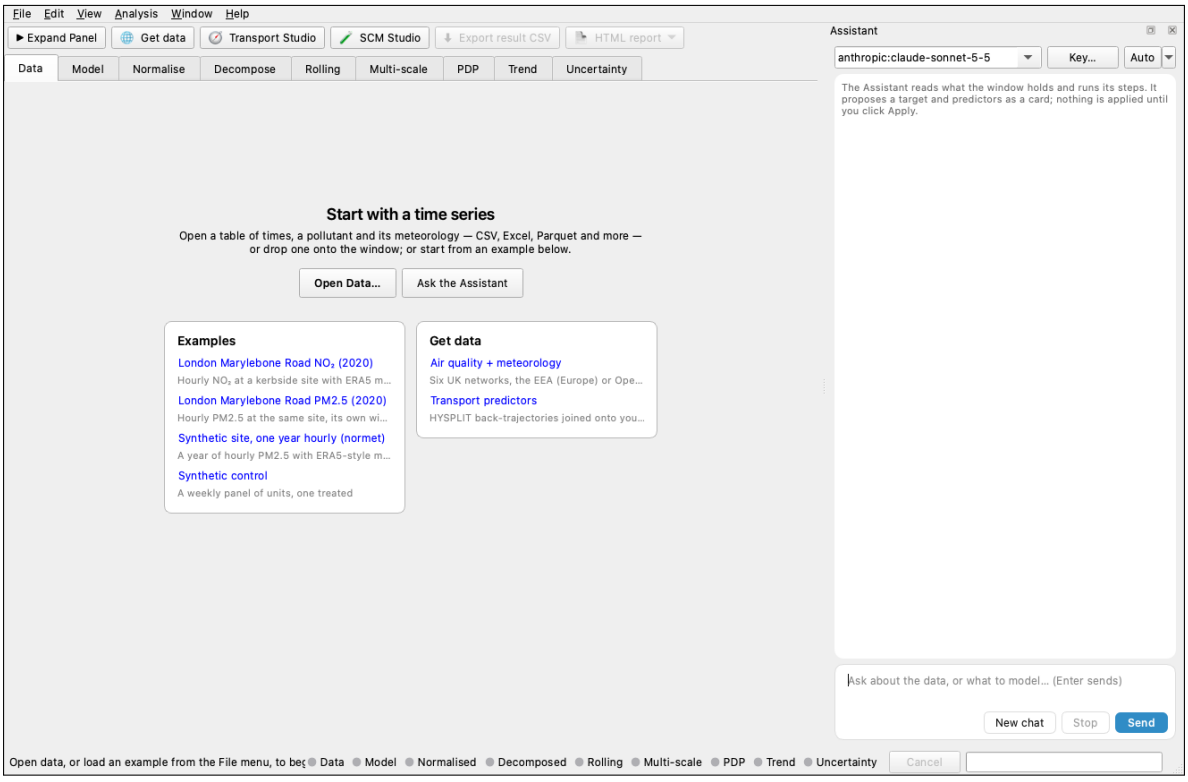



Figure 4.1: The window as it opens: the welcome on the **Data** tab, the action bar above the tabs (*Expand Panel* first), the Assistant on the right and the log at the bottom. The parameter panel is folded away until it is needed.

## 4.2 The examples

The examples are under **File → Load Example** and on the welcome's *Examples* card; each opens with its target and predictors set as its source sets them.

**London Marylebone Road $NO_2$ (2020)** The normet paper's de-weathering case study: hourly $NO_2$ at a kerbside site with ERA5 meteorology, January to August 2020, the UK's COVID-19 lockdown in it (`load_my1()` below). This manual's main-window figures are made from it.

**London Marylebone Road $PM_{2.5}$ (2020)** The paper's transport-aware case: hourly $PM_{2.5}$ at the same site, its own wind and temperature with ERA5's other fields (`load_my1_pm25()`).

**Synthetic site, one year hourly** A synthetic year of hourly data generated on the fly: $PM_{2.5}$, $PM_{10}$, $NO_2$, $O_3$, $SO_2$ and CO with ERA5-style meteorology (`u10`, `v10`, `t2m`, `d2m`, `blh`, `sp`, `ssrd`, `tcc`, `tp`, `rh2m`, `ws`, `wd`). The pollutants mix a seasonal cycle, a rush-hour cycle and a dependence on wind speed, so every step has something to show. The target is set to $PM_{2.5}$ and the meteorology ticked.

**Synthetic control panel** Opens the SCM Studio with its own example; the SCM Studio's **File → Load Example** has two (Chapter 11): the paper's London ULEZ panel and a synthetic weekly panel of 31 units, in which the unit "2+26 cities" has its $SO_2$ cut by about 35 % from 2015-10-23 and every other unit is a clean donor.

The real records of the normet model-description paper ship with the package; in a script they are loaded with:

| Loader | Contents | Used for |
|---|---|---|
| `load_my1()` | Hourly $NO_2$ + ERA5 met, London Marylebone Road, Jan–Aug 2020 | de-weathering |
| `load_scm()` | Monthly de-weathered $NO_2$, 104 UK sites, 2016–2021 | synthetic control (ULEZ) |
| `load_my1_pm25()` | Hourly $PM_{2.5}$ + met at MY1, Jan–Aug 2020 | transport-aware case |
| `load_traj_my1()` | 6-hourly HYSPLIT trajectory features arriving at MY1 | transport-aware case |
| `example_traj_dir()` | Two days of raw HYSPLIT `tdump` files | Transport Studio |

Sources: UK AURN (Defra, Open Government Licence v3.0); ERA5 (Copernicus Climate Change Service); HYSPLIT (NOAA ARL).

## 4.3 A first analysis

1. **Load data.** *Open Data…* and choose a table with its times, the pollutant and its meteorology (Chapter 12); set the *Time zone* the site keeps (Section 12.2). Bring the parameter panel out with *Expand Panel* (Ctrl+B), or ask the Assistant.
2. **Choose the columns.** Check the target and tick the predictors (Section 6.1).
3. **Train** and read the **Model** tab: is the held-out fit good enough to normalise with? (Chapter 6)
4. **Normalise** and compare the normalised series with the observed (Chapter 7).
5. **Decompose**, **Rolling** and **Partial dependence** as the question needs (Chapters 8–10); the **Trend** test says whether the series the decomposition just drew is rising, falling or flat, and the **Uncertainty** tab where the band's spread comes from (Section 8.4).
6. **Keep the result:** export the tables, an HTML report and the session as a script (Chapter 13).

# Chapter 5

# The interface

The main window has a *parameter panel* on the left and *result tabs* on the right. The panel has a page for each step of the workflow, chosen from the bar at its top — **Data**, **Model**, **Normalise**, **Decompose**, **Rolling** and **PDP** — and each step has its own *Run* button beside the settings that govern it. Results land in the tab of the same step, which comes to the front when the step finishes. The panel starts folded away, leaving the width to the results and the Assistant; *Collapse / Expand Panel* on the action bar (Ctrl+B) folds it and brings it back. The window can be narrowed to about 740 pixels with the panel folded: the action bar then wraps onto a second row.

## 5.1 The parameter panel

Its pages, and what each holds (a page's first box goes untitled, the tab naming it):

**Data** *Open Data…*, the file's name with its rows and columns, the *Time zone* its times are kept on and how they were read (Section 12.1); then *Columns*: the target and the predictors (Section 6.1).

**Model** Backend, split, training fraction, time budget (or search trials), estimators, seed; for Chronos-2 the device and fine-tuning instead (Chapter 6).

**Normalise** The variables resampled, the number of samples, CPU cores and the quantile band (Chapter 7).

**Decompose** Emission or meteorology; for meteorology the attribution and the groups (Chapter 8). Under it *Multi-scale decomposition*: three window widths (Section 9.2); *Trend*: the series to test (a part of the decomposition, its separated long-term trend, or the observed target), its seasonal period, the number of segments and how many bootstrap resamples; and *Uncertainty decomposition*: the number of ensemble members, the met draws they all share, and the members' backend (Section 8.4).

**Rolling** Window, step and samples (Section 9.1).

**PDP** Variables and grid points (Chapter 10).

The panel is as wide as its widest page needs. A configuration of the panel is saved and read from the **File** menu (Section 13.6).

## 5.2 Result tabs

| Tab | Shows |
|---|---|
| Data | overview, cycles, wind and pollution roses, statistics; rows left out |
| Model | verdict, statistics table, parity plot, feature importance, walk-forward CV |
| Normalise | observed and normalised series, the quantile band, their cycles |
| Decompose | the emission or meteorological components, the cycles |
| Rolling | the mean of the rolling windows, with their spread |
| Multi-scale | the scale-space cascade and the energy in each band |
| Trend | the tested series with its Theil–Sen line, or its segments and marked break, the verdict and the p value |
| PDP | one partial-dependence panel per variable |
| Uncertainty | the per-date spread stacked by source (weather, model, interaction) and the median's shares |

A tab with a judgement to make opens with a *verdict banner* in green (good), amber (caution) or red (not usable), followed by plain lines of what was run and, where there is one, a compact table. Every figure has a toolbar of small icons in the window's colours — home, back, forward, pan, zoom, subplot spacing, axes and curve options, save — then *For print...* (Section 13.3) and, at its right end, the value under the pointer. A figure is saved on a white page with dark ink whatever the window's appearance. Every figure is drawn as journals ask for one: 7 to 8.5 pt text, thin axes with ticks outward, legends without a frame, colours a colour-blind reader can tell apart, formulas with their subscripts ($NO_2$, $PM_{2.5}$), and a date axis written once at each level — the year once, then the months — with panels stacked on one time axis dated on the bottom panel only.

**Resolution.** The time series on the **Data**, **Normalise**, **Decompose**, **Rolling** and **Multi-scale** tabs have a *Resolution* selector above the figure: *As recorded*, *Hourly*, *Daily*, *Weekly*, *Monthly* or *Yearly*. A coarser step redraws the figure from the means of each period, which makes a long hourly record readable; steps finer than the record's own are greyed out. A record finer than daily and longer than 2,000 points opens as daily means — months of hours drawn as they are are a solid band — and the choice made on a tab is kept for its next result. Only the display changes: the result, its table and its export stay at the record's resolution. A series is drawn broken where the record has a gap (an interval over five times its usual step), not joined across it by a straight line. The $\sigma$ a figure quotes are the record's, whatever step it is drawn at.

When both an emission and a meteorological decomposition have been run on the model, the **Decompose** tab keeps both: a *Method* selector beside *Resolution* switches between them, and a saved session brings both back.

## 5.3 The action bar

Above the tabs: *Collapse / Expand Panel*; the studios side by side — *Get data* (the Data Studio, Section 12.3), *Transport Studio* (Section 12.4) and *SCM Studio* (Chapter 11); then what is done with a result — *Export result CSV* (the current tab's table) and *HTML report* (the current tab's result as a report with its provenance).

## 5.4 Run buttons

| Button | Needs | Runs |
|---|---|---|
| Train model / Load Chronos-2 | a table, one predictor ticked | `build_model` |
| Fine-tune Chronos-2 | Chronos-2 loaded | `finetune` |
| Run normalisation | a model, one variable to resample | `normalise` |
| Run decomposition | a model | `decompose` |
| Run rolling normalisation | an AutoML model | `rolling` |
| Multi-scale decomposition | an AutoML model, a normalisation | `rolling` ×3 |
| Trend test | an AutoML model (a decomposition, where it tests one) | `trend` (1 segment), `detect_trend_breaks` (more) |
| Compute PDP | an AutoML model, one variable ticked | `pdp` |
| Uncertainty decomposition | an AutoML model | `decompose_uncertainty` |

A button is enabled only when it can act, and its tooltip says what is missing when it cannot. One task runs at a time, on a background thread, so the window stays responsive; while it runs the run buttons are disabled, *Cancel* is enabled, and the status bar names the task with the time it has taken so far. Where the task says how far it has come — the rolling windows, the decomposition's steps, LightGBM's trials, the receptor times HYSPLIT runs, FLAML's time budget — the progress bar fills with it, and the line gives the share done and, once there is enough to go on, the time left; elsewhere the bar is a busy bar. When the task ends, the line its step writes says how long it took. Cancel *abandons* the task: the window is free at once, but the computation cannot be interrupted, so it finishes in the background and its result is discarded. Each step's call is added to the command log when it succeeds (Section 13.2).

A step asked for before what it needs — normalising before a model is trained, a multi-scale run before the normalisation — is answered by a *notice* above the results, not a dialog: it says what is missing and carries the step that answers it on a button (*Train model*, *Run normalisation*); × puts it away, and it goes by itself once the step has been taken. At the right of the status bar a light per step — Data, Model, Normalised, Decomposed, Rolling, Multi-scale, PDP, Trend, Uncertainty — turns green when that step has a result.

## 5.5 Menus and shortcuts

| Menu | Command | Shortcut |
|---|---|---|
| File | Open Data... | Ctrl+O |
| | Load Example (the examples, SCM's) | |
| | Get Data (AQ networks + met)... | Ctrl+D |
| | Open Recent (eight tables) | |
| | Save Results... / Load Results... | Ctrl+S |
| | Reopen Last Session | |
| | Load Session Script... | |
| | Methods Paragraph... | |
| | Export Current Result... | Ctrl+E |
| | Export HTML Report: English, Chinese | |
| | Save Session as Script... | |
| | Save Config... / Load Config... | |
| | Run History... | |
| View | Command Palette... | Ctrl+K |
| | Collapse / Expand Panel | Ctrl+B |
| | Log..., Command Log... | |
| | Assistant | Ctrl+Shift+A |
| | Appearance: Follow System, Light, Dark | |
| | Language: Follow System, English, Chinese | |
| Analysis | Synthetic Control (SCM) Studio... | Ctrl+Shift+S |
| | Polar Plot... | |
| Help | User Manual | F1 |
| | Keyboard Shortcuts... | |
| | Documentation, About Normet | |

On macOS, Ctrl is the Command key. The *command palette* (Ctrl+K) lists every menu command and every result tab; type a few letters of one and press Return. **Help → Keyboard Shortcuts** lists every shortcut, read from the menus; **Help → User Manual** (F1) opens this manual in the window's language.

## 5.6 The log and the Assistant

**View → Log...** opens normet's log in a window of its own, which can stay open beside the main window: what each step did, how many rows, FLAML's chosen estimator and its hyperparameters, how many windows a rolling run made. **View → Command Log...** opens the calls that repeat the session (Section 13.2). When a task fails, its full traceback is written to the Log and its last line shown in a dialog. The *Assistant* takes the right of the window, its whole height (Chapter 14).

## 5.7 Appearance and language

The window and its figures follow the system's light or dark appearance; **View → Appearance** pins one. A figure on a dark window is recoloured for display only — its faces, text, ticks and grid; colours that carry data are left alone — and saved in its light colours.

The window speaks English or Chinese: the system's language by default, or the one **View → Language** chooses, kept for the next session. The switch is immediate — menus, buttons, tabs, tooltips, verdicts and the figures' titles, labels and month names are written again in the language chosen, and a figure is saved as it is shown. The data's own words (column names, site codes), the log, the command log's code and the Assistant's conversation stay as they were written. What the system draws itself stays in the system's language whichever the window is

written in: the open and save panels, their titles too, and, on a Mac, the items macOS adds to the menus (*AutoFill*, *Start Dictation*, *Emoji & Symbols*). An HTML report (Section 13.2) is written in the language picked as it is exported — **File → Export HTML Report**, or the *HTML report* button, lists each language named in itself — its headings, labels and figure; the language is kept for the next report, and is the system's until one is picked.

# Chapter 6

# Training a model

## 6.1 The table and its columns

A table opened in the main window needs a column called `date` (any case) whose values parse as dates; every other column numeric is offered as a target and as a predictor. The target is preset to the first column named like a pollutant ($PM_{2.5}$, PM10, $NO_2$, O3, SO2, CO, NOx), and the predictors to the columns recognised as meteorology:

```
u10 v10 d2m t2m blh sp ssrd tcc tp rh2m ws wd temp rh pressure
```

(matched without regard to case). *All* ticks every predictor — and, once all are ticked, unticks every one — and *Met only* ticks the meteorology alone. The target is never a predictor, even when ticked. The four calendar variables are added to every model automatically and are not listed.

**Note.** A meteorological column under another name is not recognised and must be ticked by hand — in MY1, `atmos_pres`. The same applies to the **Normalise** page's *Met only* (Chapter 7).

The **Data** tab has four pages. **Overview** shows the record at a glance: drag across the plot to pick a span, or click it to pick the point under the cursor (the period it stands for, where the series is averaged), and the rows are listed below the plot; with Shift held (or Ctrl, or Cmd) a drag or a click adds to the pick, a click on a picked point drops it, and Escape clears them. One line under the plot says the pick — its rows, and how many of them are left out — at its left, and the time, the variable and the value under the cursor at its right. In the list, rows are selected as in a spreadsheet — Ctrl- or Cmd-click for rows apart, Shift-click or a drag down the rows for a run — and right-clicked, on them or on their numbers, to exclude or restore them (a row right-clicked that is not selected is selected alone first). A right-click on the plot offers the same for every row picked, a point outside the pick picked first; *Exclude picked* and *Restore picked*, on the plot's *Resolution* row, do the same for the rows selected, or every row picked when none is. The list stays, its excluded rows in grey, to go on from. The *Exclude* box beside them takes spans typed one to a line (`2020-03-23 .. 2020-04-05`, or a single date) and *Apply*. Rows left out are shaded on the plot and are left out of every step after: training, normalisation, decomposition and the rest. The Command Log writes them as `left_out` and `df_kept`, and the session keeps them. **Cycles** draws a column's diurnal, weekly and seasonal cycles; **Wind**, when the table has wind speed and direction, the wind rose, a pollutant's pollution rose and its polar plot; **Statistics** each column's values, missing share, mean, standard deviation, minimum, quartiles and maximum. All four read the rows kept.

## 6.2 Settings

**Backend** `flaml`, `lightgbm` or `chronos-2` (Section 6.4). Chronos-2 is greyed out when its extra is not installed, its tooltip saying why.

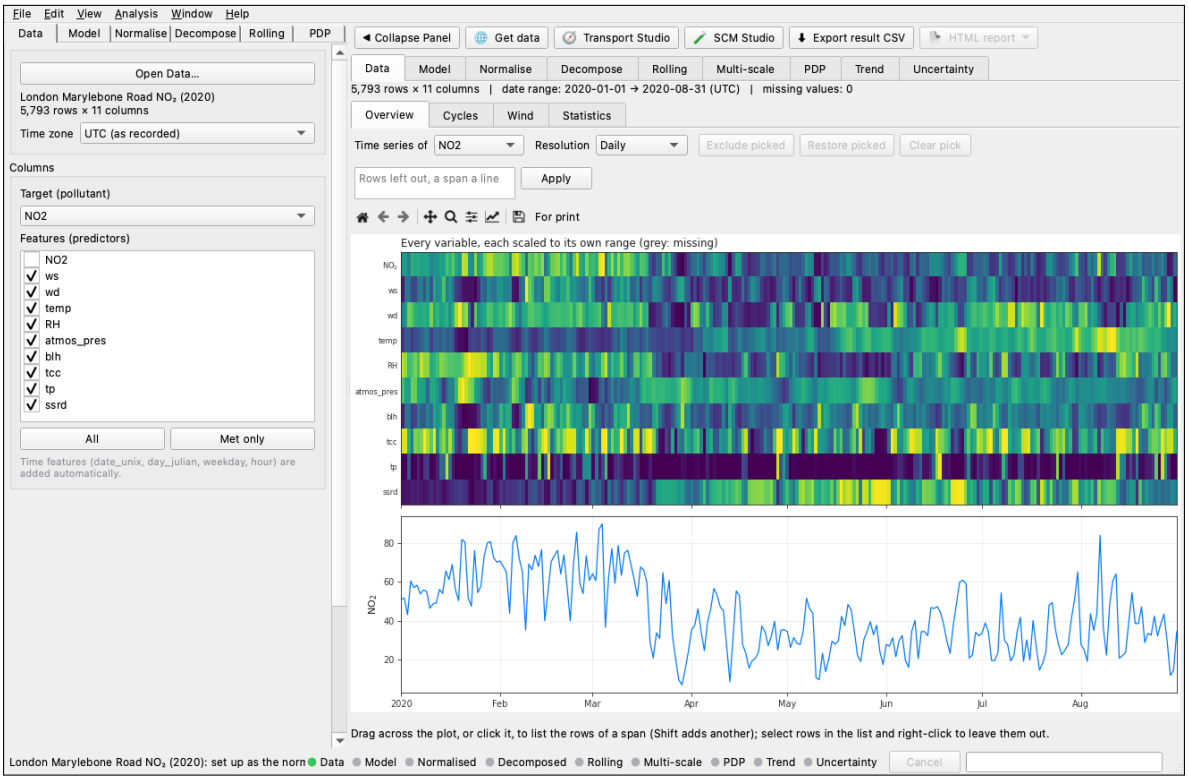



Figure 6.1: MY1 opened from **File** → **Load Example**: 5,793 hourly rows of 11 columns from 2020-01-01 to 2020-08-31, with no missing values. The example sets the paper's target, $NO_2$, and its nine meteorological predictors (`atmos_pres` among them, a name the window does not recognise). The **Overview** page draws every column as a heatmap, each scaled to its own range, above the series picked (here $NO_2$, daily).

**Split method, Training fraction** Section 3.1.1; 0.75 by default.
**Time budget** FLAML's search time, 60 s by default. For LightGBM the row becomes *Search trials*, 20 by default; each backend's value is kept when you switch.
**Estimators** (FLAML only) LightGBM alone by default; add more to widen the search.
**Seed** 7654321 by default: the split and the search are seeded with it.

## 6.3 Reading the Model tab

*Train model* runs `build_model` — preparation, the split and the search — and fills the **Model** tab (Figure 6.2). The verdict is read from the held-out $R^2$: 0.7 or above is a good fit, 0.5 to 0.7 moderate, below 0.5 poor. The table is `modStats` for the training rows, the testing rows and all rows (Section 3.2.1); the parity plot shows the held-out rows against the prediction, square, with the 1:1 line and, in its corner, the number of rows, the slope of predicted on observed (below 1: the extremes are pulled in) and the mean bias — as a density of hexagons where the rows run to thousands; the bars are the model's feature importance (split counts for LightGBM). Under the table, one line reads the point forecast's *CRPS reference* — $\sqrt{\pi/2} \cdot \mathrm{MAE}$ over the held-out rows, the point forecast's score on the scale of the continuous ranked probability score. A forecast that carries a spread scores its own CRPS, and only a score below this reference says the spread is worth carrying.

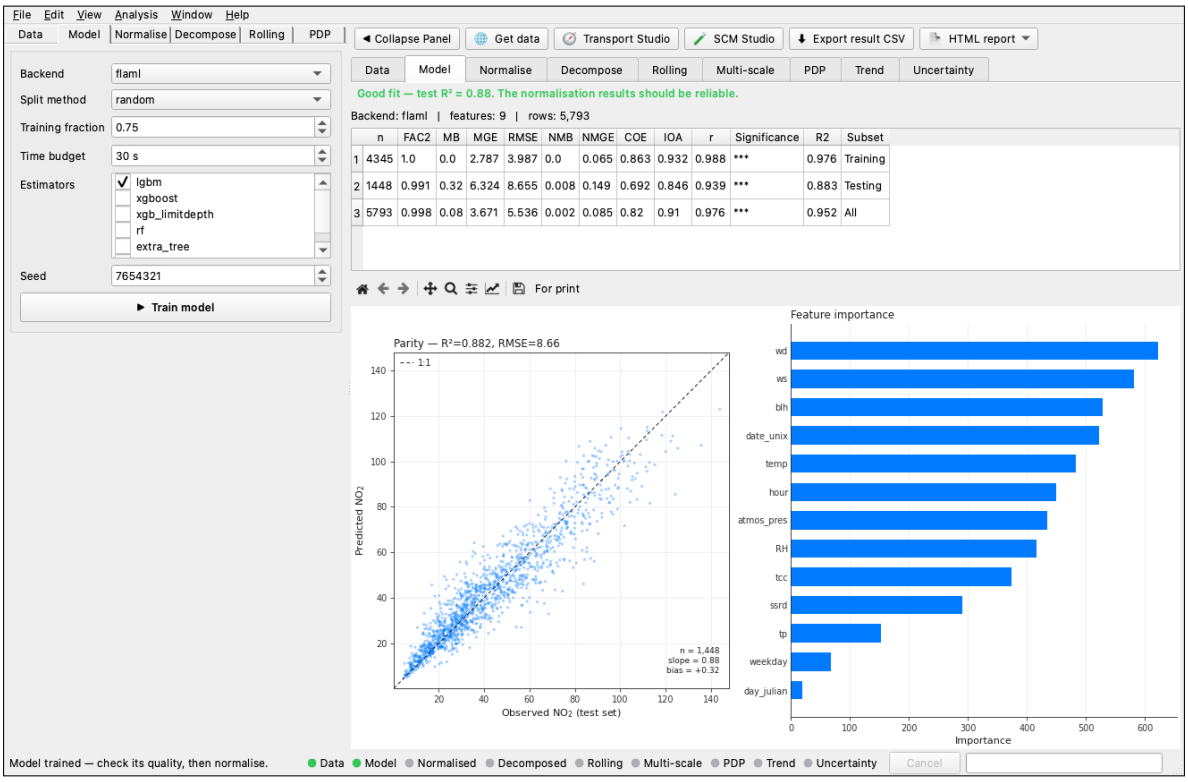



Figure 6.2: MY1 trained with FLAML (LightGBM, 30 s budget, random split): on the 1,448 held-out hours $R^2 = 0.883$ and RMSE $= 8.66\ \mu\mathrm{g\,m^{-3}}$, against 0.976 on the 4,345 training hours. Wind direction and speed, the boundary-layer height and the trend carry the most splits.

A fit on the training rows much better than on the held-out rows is expected of boosted trees and is not by itself a problem; a poor held-out fit is. Before normalising with a moderate or poor model, try more predictors, a longer budget or more estimators. The normalised series inherits the model's errors.

A single random split flatters the model: every test hour's neighbours sit in the training rows,

so an autocorrelated series can be forecast locally without being forecast well. *Walk-forward CV* checks that number: the same backend retrains on an expanding window and scores the block of hours immediately after it, five times by default (*CV folds*); it takes the training budget once per fold. The tab gains a *View* selector between the split's figures and the folds — a bar of $R^2$ per fold with the split's value drawn across the bars, and a table of each fold's test window, its size and its $R^2$, RMSE and mean bias. When the split's line sits well above the bars the verdict says so: trust the folds. For a series whose regime drifts, where a random split samples the future as often as the past, the folds are the honest number.

## 6.4 Chronos-2

With `chronos-2` chosen the training controls are hidden — there is nothing to train, no budget, no split — and the button reads *Load Chronos-2*. The first load downloads about 500 MB of weights. *Device* chooses where it runs (*auto*: CUDA, else Apple Silicon, else CPU). At least one meteorological predictor must be ticked: without covariates the model makes a plain forecast, not a de-weathering.

The **Model** tab then reports the checkpoint, device, context and horizon, and the covariate-sensitivity check (Section 3.9): the mean and largest shift of the forecast's median when the future weather is shuffled, and that shift as a percentage of the prediction. The verdict is green at 5 % or more, amber from 1 to 5 % (the series is mostly carried by the target's own history) and red below 1 % (the model is autoregressing, and a de-weathered series from it would be meaningless).

The table also carries a quantile backtest: the checkpoint forecasts the last horizon once at the 10/50/90 percentile levels, the band is scored against the observed tail by CRPS (the CDF interpolated through the three levels), and its median is scored as a point forecast ($\sqrt{\pi/2} \cdot$ MAE again). The line under the table says whether the band beats its own median's reference — whether the spread is worth carrying — and the pinball losses break the score down by level. The line names the window it speaks for: one horizon is a quick check, not the backtest one would quote.

*Fine-tune Chronos-2* adapts the loaded checkpoint to the site's own record: *lora* trains small adapters and leaves the base weights alone (it needs `peft`); *full* updates all 119 million parameters and overfits a single site more easily. *Steps* sets the optimiser steps (300). The tuned weights replace the model for the session; normalise again to use them, or load Chronos-2 again to go back. On the Chronos-2 backend, rolling, partial dependence and the emission decomposition are disabled, and the multi-scale decomposition, built on rolling, does not apply (Section 3.9).

# Chapter 7

# Normalising

**Resample vars** Every variable the model was trained on, the four calendar variables included. Ticked variables are resampled (their influence averaged out); unticked ones keep their observed values. After training the list is set to *Met only* — the usual de-weathering — and *All* ticks everything, which removes the trend and the seasonal and weekly cycles as well.

**Samples** Monte-Carlo draws, 300 by default (8 on Chronos-2; Section 3.3 on how many are enough).

**CPU cores** *auto* uses all but one.

**5–95 % quantile band** Also return the 5th, 25th, 75th and 95th percentiles of the draws and shade the outer band.

Beneath the series, the observed and normalised series are averaged by the hour of the day, the day of the week and the month, each mean with its 95 % confidence interval — the record's cycles, as openair's `timeVariation` draws them, and how much of each the weather moved. A cycle the record cannot show (the hours of a daily record) is left out; the cycles are averaged from the record as it is, whatever resolution the series above is shown at. In a script: `nm.cycle_profiles(result, ["observed", "normalised"])`.

The line above the figure gives the number of timestamps, draws and resampled variables, and the mean of normalised − observed over the record: here $-0.26\ \mu g\,m^{-3}$, since over eight months the weather averages out. The monthly means tell the story the figure shows:

| $NO_2$ ($\mu g\,m^{-3}$) | Jan | Feb | Mar | Apr | May | Jun | Jul | Aug |
|---|---|---|---|---|---|---|---|---|
| observed | 60.3 | 65.2 | 51.7 | 34.5 | 30.7 | 35.1 | 31.6 | 36.6 |
| normalised | 59.7 | 64.7 | 54.0 | 34.7 | 32.1 | 32.5 | 32.1 | 33.8 |

From February to April the normalised series falls by 30.0 $\mu g\,m^{-3}$ (46 %), against 30.7 $\mu g\,m^{-3}$ observed: almost none of the spring drop was the weather. In March the weather was cleaner than usual (observed 2.3 below normalised), in June and August dirtier.

**Note.** On Chronos-2 the first `context_length` rows are seeded with the observed values; they are shaded grey on the figure, the line under it says how many there are, and the mean is taken over the rest.

**Caution.** The normalised series is the model's expectation under typical weather. A model that has not learnt the weather's effect — a poor fit, or meteorology missing from the predictors — returns something close to the observed series, which looks like a de-weathered series with nothing to remove. Read the **Model** tab first.

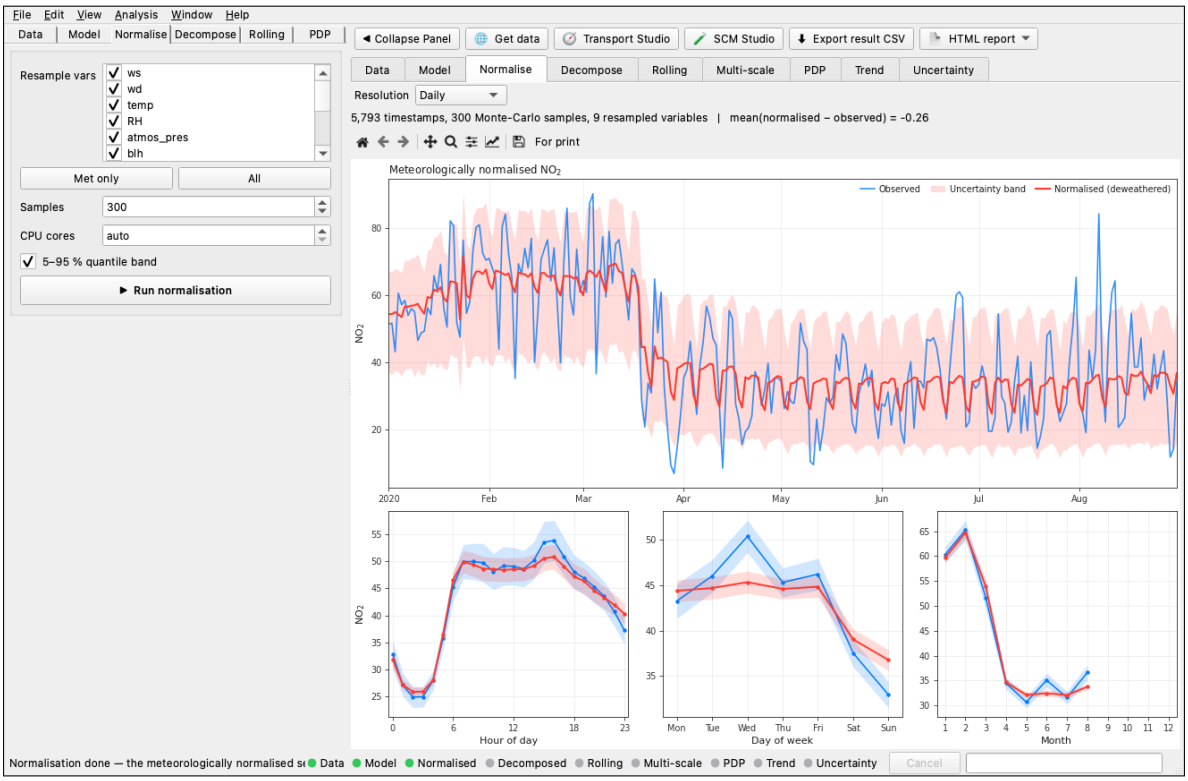



Figure 7.1: MY1 normalised with 300 draws of all nine meteorological variables, shown as daily means. The normalised series (red) falls in the third week of March, the lockdown, which the weather does not explain. Beneath, its cycles: the normalised series keeps the rush hours and the weekday–weekend difference — the calendar is held at its observed values — with less of the observed series' spread.

# Chapter 8

# Decomposing

## 8.1 By time scale: the emission decomposition

With *Method* set to *emission*, the decomposition runs the nested normalisation of Section 3.4 and plots the normalised series with its trend, seasonal, weekly, diurnal and short-term components, each with its standard deviation $\sigma$ (Figure 8.1). The line above the figure names the component with the largest $\sigma$. On a record finer than daily, the diurnal and weekly components are drawn on the right as their cycles — the component's mean by hour of day and by day of week, shaded between its 25th and 75th percentiles — rather than as months of oscillation.

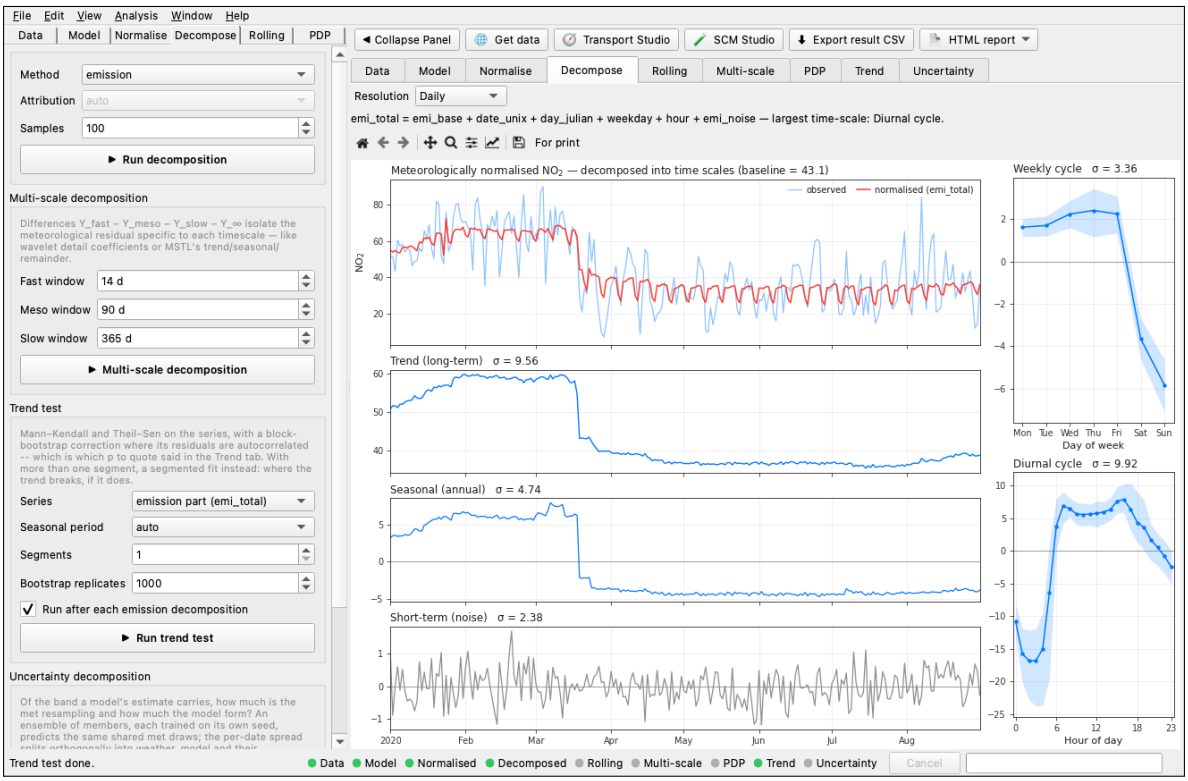



Figure 8.1: The emission decomposition of MY1 (100 draws per normalisation; baseline 43.1 $\mu g\,m^{-3}$). The lockdown appears as a step in both the trend ($\sigma = 9.56$) and the seasonal component ($\sigma = 4.74$): a one-off event is not a season, and the record is too short to tell them apart. The diurnal cycle (right) is the largest ($\sigma = 9.92$): low before 05:00, high through the working day, its spread wide because it shrank after the lockdown, as rush-hour traffic did; the weekly cycle ($\sigma = 3.36$) dips at the weekend.

## 8.2 By variable: the meteorological decomposition

With *Method* set to *meteorology*, the decomposition splits observed − normalised among the meteorological variables (Section 3.5). *Attribution* is enabled and *Groups* appears (it is hidden for *emission*):

**Attribution** *sequential*, *shapley*, or *auto* (Shapley when groups are given, else sequential).

**Groups** A table of the meteorological variables the model was trained on, each with its group: *Wind*, *Temperature and humidity*, *Boundary layer and pressure*, *Sky and rain*, or a name typed in. *Suggest groups* puts each recognised variable in its group; *Each alone* clears them. A variable left without a group is a contribution of its own. A group's rows are tinted in one colour, and the line under the table says how many contributions the grouping makes.

The figure shows met_total and one panel per variable or group, with a ranking of their $\sigma$ beside `met_noise`, the part the model does not reproduce (Figure 8.2).

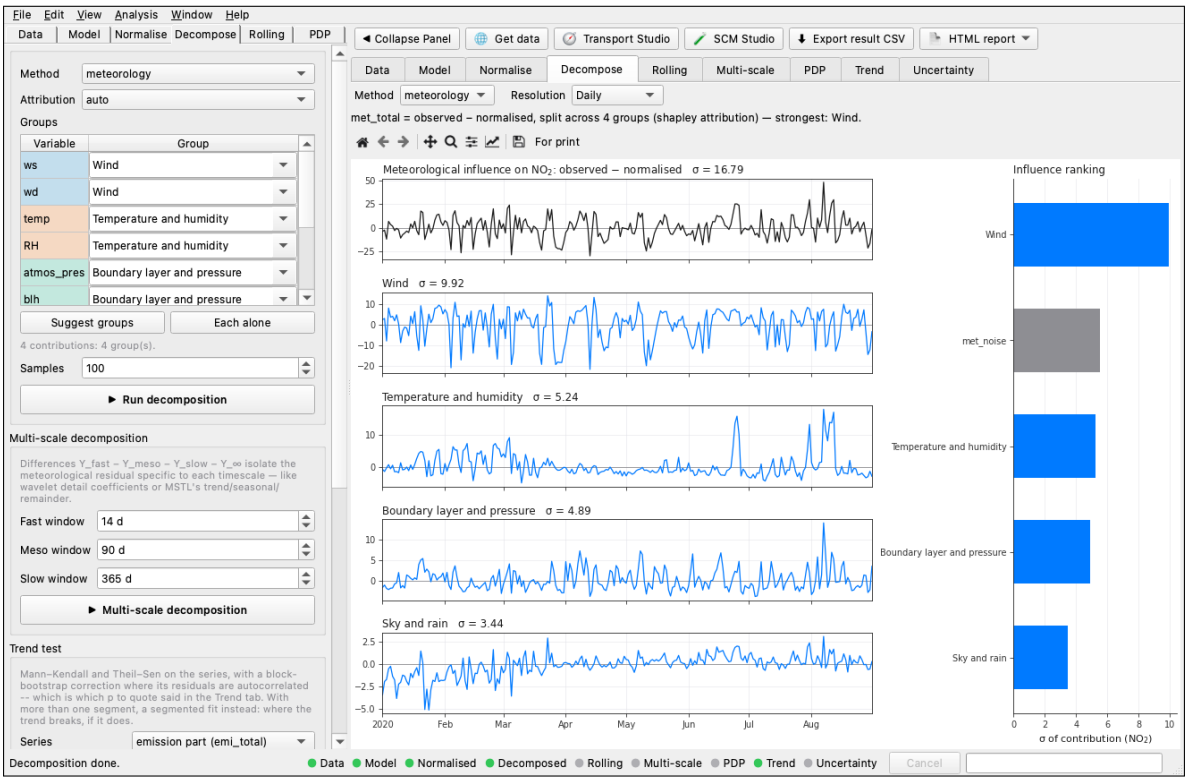



Figure 8.2: MY1's weather influence (observed − normalised, $\sigma = 16.79\ \mu g\,m^{-3}$) split among four groups by exact Shapley values (16 coalitions): wind (`ws`, `wd`; $\sigma = 9.92$), thermodynamics (`temp`, `RH`; 5.24), mixing (`blh`, `atmos_pres`; 4.89) and sky (`tcc`, `tp`, `ssrd`; 3.44).

The decomposition runs $k + 1$ normalisations sequentially or $2^k$ by Shapley, each of *Samples* draws, so its cost grows quickly with the number of variables. Ten variables by Shapley is 1024 normalisations: group them. Unlike the normalisation, the meteorological decomposition always resamples every meteorological variable trained on; it does not follow the **Normalise** page's list.

## 8.3 Trend

The question a decomposition invites: is the series it just drew rising, falling or flat, and is the rise real? The *Trend test* box under *Decompose* runs Mann–Kendall and Theil–Sen on the chosen series and, the record carrying the autocorrelation the correction exists for (MY1's emission part:

AR(1) residual acf 0.94), replaces the slope's classical error bar with a moving-block bootstrap. It runs again each time an emission decomposition ends, if *Run after each emission decomposition* is ticked.

*Series* names which one is tested: the decomposition's emission part (the default: what the weather does not explain), its *separated long-term trend* — the long-term component Section 3.4 drew, cycles already separated into their own components, so the cleaner input to a trend test — its weather part, or the observed target itself. *Segments* names how many: one (the default) is the single-slope test just described; two to four run the *segmented fit* instead — `detect_trend_breaks` on the chosen series. It splits the record where the trend breaks, if it does, each segment reported with its own Theil–Sen slope and interval, and the break accepted only under the two-part gate (Section 3.4 for the components it tests on): the BIC must improve, and the SSE improvement of the final fit over the single slope must beat the same search re-run on block-bootstrapped resamples (the p to quote). A record that fails a gate is reported as one regime, with the best candidate cut and its p kept so the tab can say what was looked at. Raising *Segments* above one moves a still-default *emission part* choice to the long-term trend (the clean input); the choice stays until changed.

The **Trend** tab (Figure 8.3) for one segment shows the series with its Theil–Sen line and a verdict that says which of the two p values to quote: the classical Mann–Kendall one is the anti-conservative one on an autocorrelated record, and the record is autocorrelated. For two or more segments it shows the series with its *piecewise* Theil–Sen lines and a marked break, a verdict that says whether the search found a break (where, and the block-bootstrap p to quote) or that the gates refused one, per-segment rows in the table (start, end, n, slope, 95 % interval, per year), and the same CSV export and command-log entry as the one-segment run (`trend_breaks_call` in the script, not `trend_call`) (Figure 8.4).

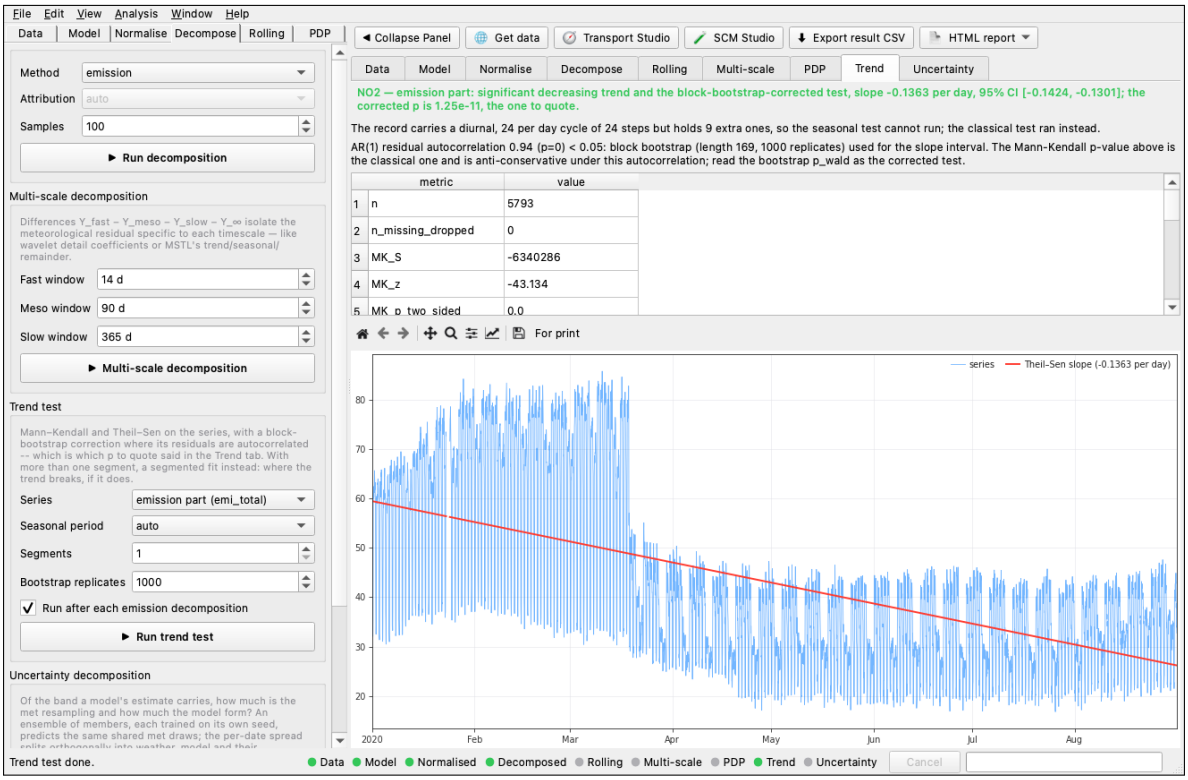



Figure 8.3: The trend test of MY1's decomposition's emission part (Section 3.4): the series with its Theil–Sen line and the two p values — the classical Mann–Kendall one and, the record being autocorrelated, the block-bootstrap one — with the verdict above the figure saying which to quote.

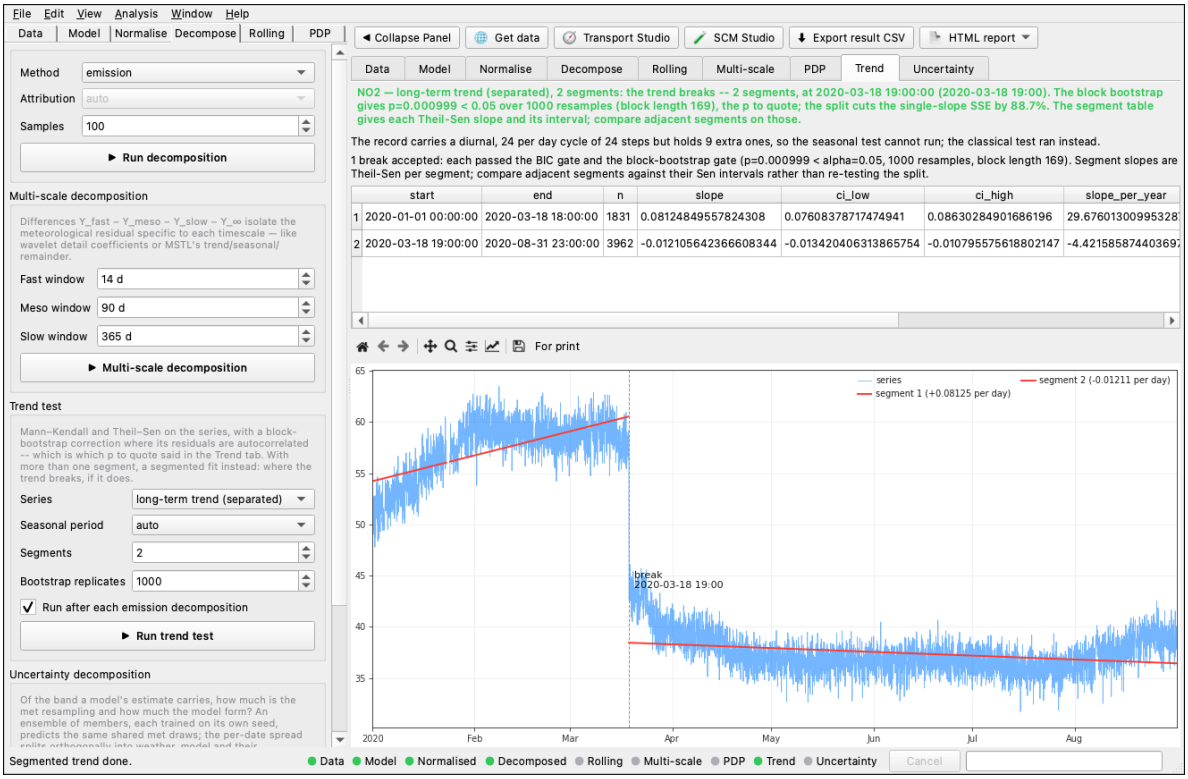



Figure 8.4: The segmented fit (Segments 2) on MY1's separated long-term trend: the search found the break at 2020-03-18 19:00 — the 2020-03-23 lockdown's lead-in — the single-slope SSE down 88.7 % with the block-bootstrap p of the search 0.001, the one to quote. Segment 1 runs +0.0812 per day (95 % [0.0761, 0.0863]); segment 2 flattens to −0.0121 per day (95 % [−0.0134, −0.0108]). The piecewise Theil–Sen lines and the marked break are in the figure; the per-segment table and the verdict above it are the Trend tab's.

## 8.4 Uncertainty decomposition

The question a de-weathered estimate invites: of the band around an effect, how much is the met resampling and how much is the model form? The *Uncertainty decomposition* box under *Trend* answers by attribution, not by another Monte-Carlo run. It trains *Models* (default 4) members, each on its own seed and on the table as loaded, from the trained variable set; every member then predicts the same *Samples* (default 300, the variables ticked on the **Normalise** page) met resamples, the draws seeded once by the window's seed so that draw $j$ is identical across members. Each timestamp's (members × draws) prediction matrix splits its variance orthogonally into the between-draw term — the weather variance — the between-member term, the model-form variance, and the interaction: what a member reacts to in a draw that the ensemble on average does not. The three summands sum to the total per-date variance exactly — an identity the package pins in a test — so the attribution cannot quietly stop adding up. The *Member backend* chooses how each member is trained: `lightgbm` (the default: a fast random search with the Model page's budget as its trials) or `flaml` (an AutoML search per member, at that member's budget each).

The **Uncertainty** tab (Figure 8.5) stacks the per-date variance, weather under model under interaction, the line over the stack the total, and says the headline above the figure: the median share of the total by source. For MY1, three members and 50 shared draws under the figure's settings, the median spread is 96 % met resampling, 3 % the interaction and 0 % model form — the fitted model is nearly certain about what the weather does, and the band an effect is

tested against is the weather's, not the model's. The table gives each source's median variance and median share; the shares are medians across the record, so a single extreme day does not re-weight them. The run is added to the command log as the `decompose_uncertainty` call the window made, and a saved session brings the run back whole, its figure re-drawn. The cost is *Models* fits and *Models* normalisations of *Samples* draws — the same number of predictions as one `do_all_unc` run, and no more; a run of MY1's settings takes a minute or two, not an hour.

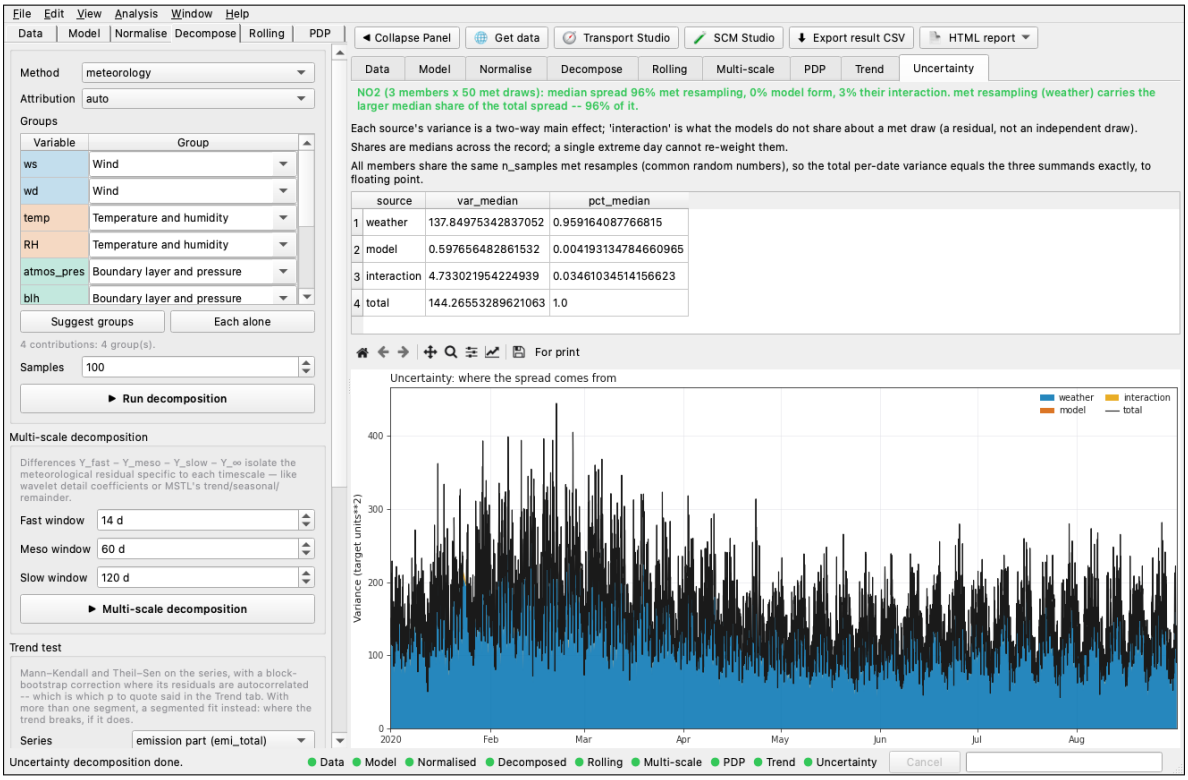



Figure 8.5: The attribution of MY1's per-date spread: the stacked bands (met resampling, model form, the interaction) total 144.3 $(\mu g\,m^{-3})^2$ of variance in the median, and the verdict above says where it goes — 96 % met resampling, 3 % interaction, 0 % model form (three members sharing 50 met draws, lightgbm members). An effect smaller than that band is not separated from the weather by the model form at all.

# Chapter 9

# Rolling and multi-scale

## 9.1 Rolling normalisation

*Window* (14 d), *Step* (7 d) and *Samples* (100) set the windows of Section 3.6; the variables resampled are the **Normalise** page's. The tab shows the mean of the overlapping windows at each time, with ±1 standard deviation across them where two or more overlap (Figure 9.1). The result table, exported, has one column per window.

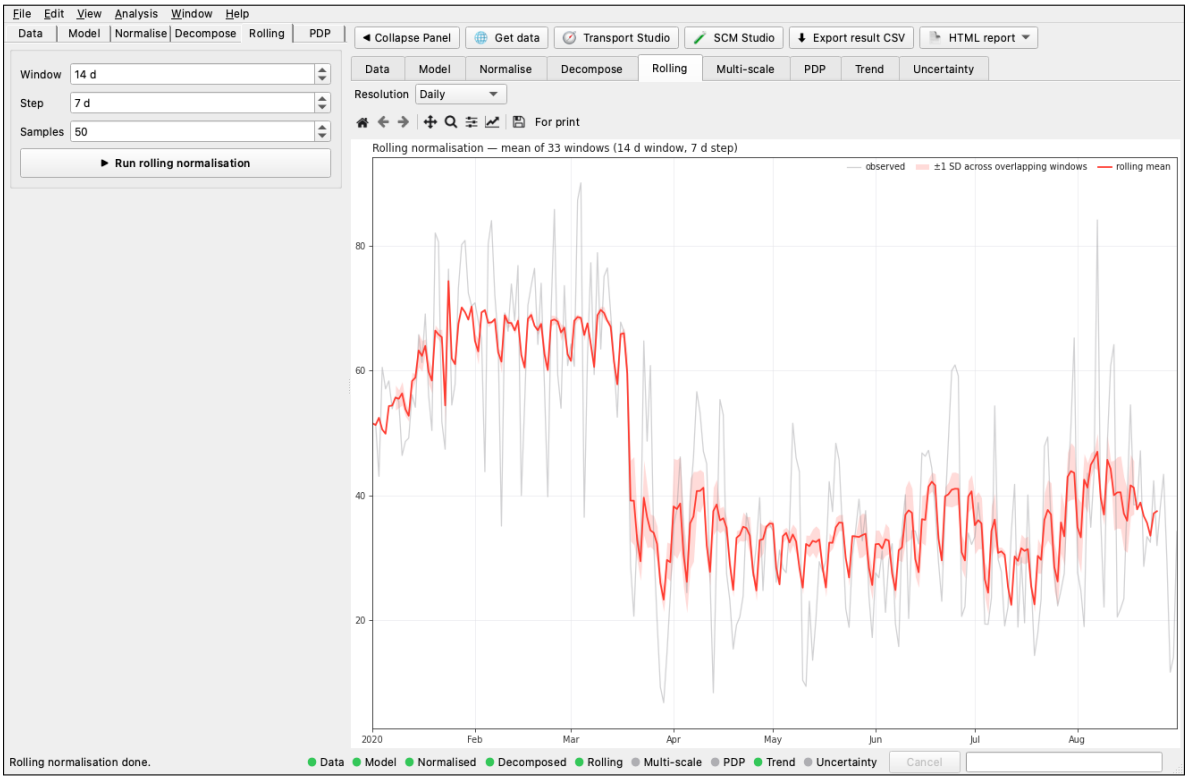



Figure 9.1: Rolling normalisation of MY1: 33 fortnightly windows started weekly, 50 draws each. Each window removes only the weather that varies within its own fortnight.

## 9.2 Multi-scale decomposition

The multi-scale box runs rolling normalisations at three widths — *Fast* (14 d), *Meso* (90 d) and *Slow* (365 d) by default — and differences them against each other and against the normalised series (Section 3.7); so the normalisation must have been run first. It uses the **Rolling** page's

*Samples* and costs about three rolling runs. The verdict names the band with the most energy ($\sigma$); a band that could not be computed is named in the notes, with why.

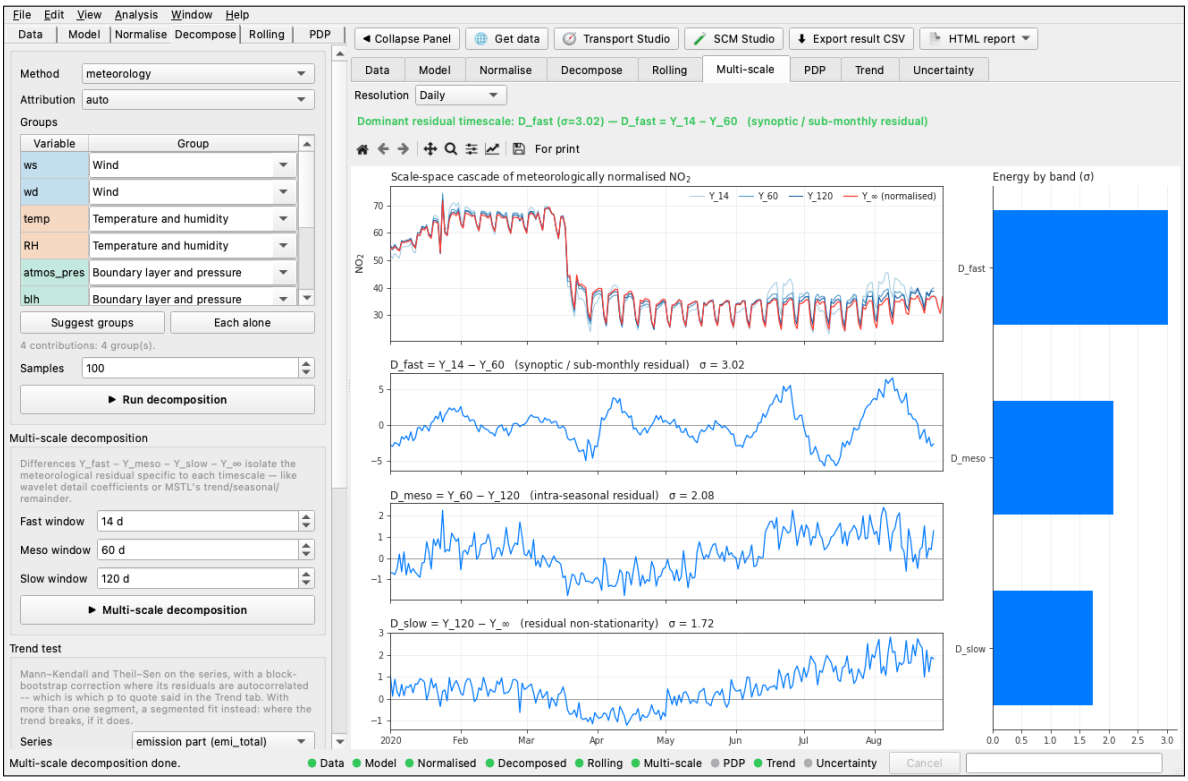



Figure 9.2: Multi-scale decomposition of MY1 at 14, 60 and 120 days (the 365-day default is longer than this eight-month record): 77, 13 and 5 windows. The synoptic band carries the most ($D_{\text{fast}}$, $\sigma \approx 3.0$), then the intra-seasonal and the slow.

# Chapter 10

# Partial dependence

After training, the variable list holds every predictor and the calendar variables, with the first four ticked. *Time variables* and *Met only* toggle those groups; *Grid points* (50) sets the resolution. Each panel shows the curve (`pdp_mean`) with a band of ± `pdp_std`, the spread of the predictions across the training rows at that value (Section 3.8). The short ticks along the bottom are the variable's deciles in the training data: where they are far apart — a curve's ends — the data are few. A variable of a few whole values (the hour, the weekday) is drawn as steps, a point on each value.

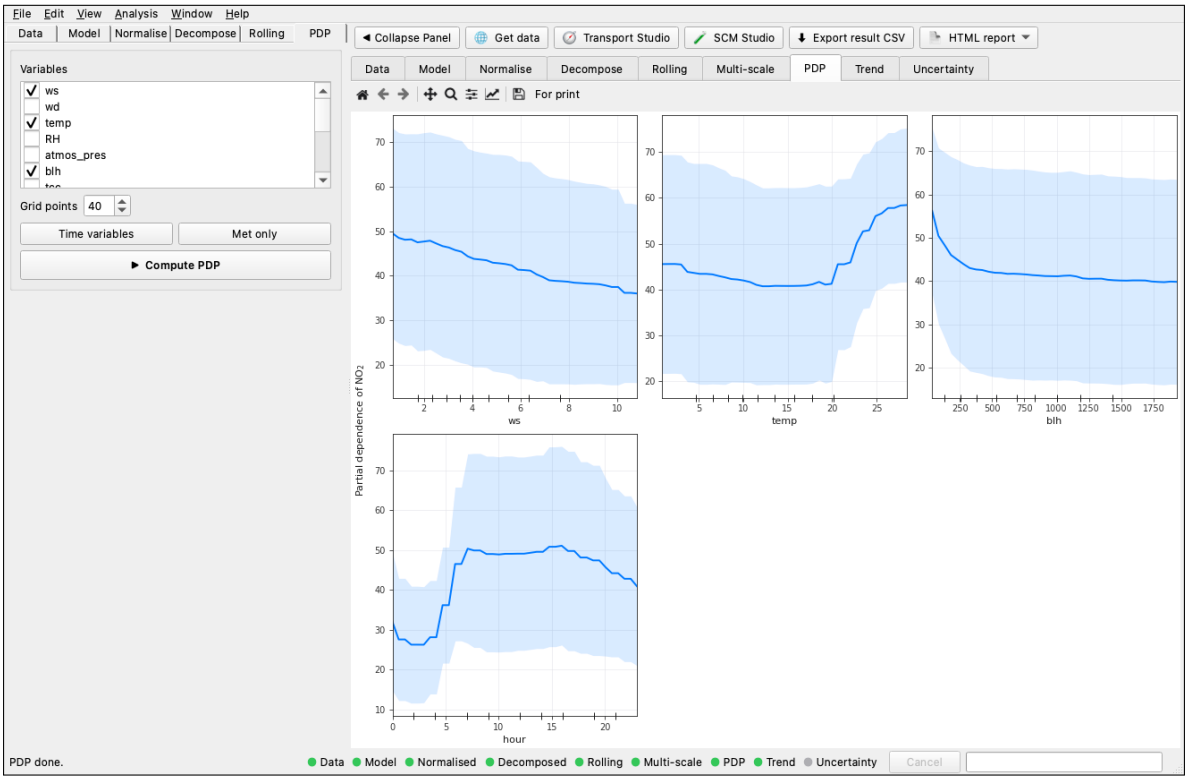



Figure 10.1: Partial dependence of the MY1 model: $NO_2$ falls as wind speed rises and as the boundary layer deepens from its shallowest, rises above about 20 °C, and follows the traffic day — low before 05:00, high from 07:00 to 17:00.

The bands are wide because the other predictors still vary across the rows; they are not a confidence interval for the curve.

# Chapter 11

# Synthetic control (SCM Studio)

The synthetic-control workflow takes a different input — a long panel of many units — so it has a window of its own, opened from *SCM Studio* on the action bar, **Analysis** → **Synthetic Control (SCM) Studio** (Ctrl+Shift+S) or the welcome's example card. It is laid out as the main window is: the panel on the left, its pages chosen from a bar — **Data** (the panel data and its columns), **Design**, **Estimator**, **Inference** — result tabs on the right (**Panel**, **Fit**, **Fit Checks**, **Placebo Space**, **Placebo Time**, **Uncertainty**, **All Units**, **Similarity**), and *Export current tab data* above them. The method is described in Section 3.10.

This chapter's figures reproduce the ULEZ case study of the normet paper: the London Ultra Low Emission Zone, launched on 8 April 2019, evaluated at Marylebone Road (MY1) on monthly de-weathered $NO_2$, with traffic sites elsewhere in the UK as donors. To keep COVID-19 out of the post-period the panel ends in February 2020, and only traffic sites with a complete record are kept: 53 donors over 38 months. The SCM Studio's **File** → **Load Example** → **London ULEZ** sets this design up; in a script:

```
import normet as nm
scm_df = nm.datasets.load_scm()
df = scm_df.query("'2017-01-01' <= date < '2020-03-01'")
df = df[df["type"].str.contains("Traffic")]
counts = df.dropna(subset=["NO2_dw"]).groupby("code").size()
donors = [c for c in counts[counts == df["date"].nunique()].index if c != "MY1"]
df = df[df["code"].isin(donors + ["MY1"])]
df.to_csv("ulez_panel.csv", index=False)
```

## 11.1 Panel data and columns

*Open Data...* (also **File** → **Open Data...**) reads a long table, in any of the formats of Section 12.1: one row per unit and date. *Use main-window data* takes the main window's table, when it has a unit column. The *Date* column is guessed from its name, the *Unit* column as the first non-numeric column with between 3 and a quarter as many distinct values as rows, and the *Outcome* as a numeric column whose name ends in `wn` (the de-weathered series of the paper's tables), else the first numeric one. Check all three.

## 11.2 Design

**Treated unit** The unit the intervention reached.

**Cutoff date** The first date of the post-period. The month containing the intervention is a judgement: here 2019-04-01, the ULEZ's launch month.

**Donor pool** Every other unit, ticked. Untick any unit the intervention could have reached, directly or by spill-over — a donor that was itself treated biases the counterfactual towards the treated unit.

**Rank donors by similarity** (needs the `foundation` extra) Embeds every unit's pre-cutoff record with Chronos-2 and ranks the others by the cosine similarity of their dynamics to the treated unit's, on the **Similarity** tab; *Tick the top N* and *Apply* make those the pool.

The **Panel** tab plots the outcome of every unit with the treated one in red, and checks the panel: amber when more than 10 % of the date × unit cells are missing or there are fewer than 10 pre-cutoff dates, green otherwise.

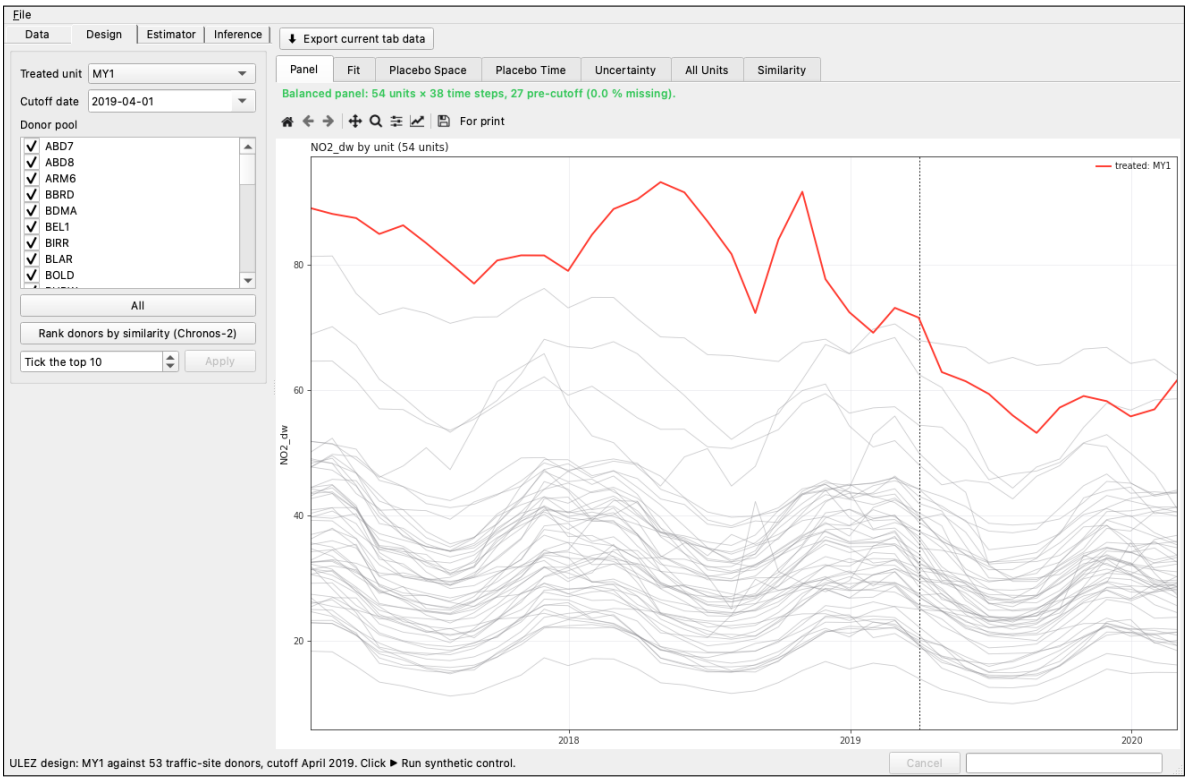



Figure 11.1: The ULEZ panel: MY1 (red) against the 53 traffic-site donors, monthly de-weathered $NO_2$, January 2017 to February 2020. MY1 is the highest of them throughout.

## 11.3 Fitting

*Backend* chooses the estimator (Section 3.10); *ML backend* and *Time budget* (15 s) apply to `mlscm`, *Draws* (1000) to `bayesian`. *Run synthetic control* fills the **Fit** tab: observed and synthetic, the effect, the largest donor weights, and the diagnostics table.

For `scm`, `abadie`, `did`, `mcnnm` and `robust` the fit is followed by one more, of the same design with the treated unit's pre-cutoff values shuffled in time: a series with the treated unit's level and spread but no relation to the donors (Section 3.10.2). Its pre-cutoff $R^2$ is the table's `pre_r2, shuffled` row, and it decides how the verdict reads:

- If the shuffled series also reaches $R^2 \geq 0.5$, the design fits anything and the observed $R^2$ is not read. The verdict is amber, gives the ATT and both $R^2$, and sends you to Placebo in space and the other estimators.
- Otherwise the verdict is read from the pre-cutoff $R^2$: green at 0.8 or above, amber from 0.5, red below; it gives the ATT.

`mlscm` and `bayesian` are too slow to fit twice; their verdict is read from the pre-cutoff $R^2$ alone. An `scm` fit adds a line saying its weights balance the ridge residuals only.

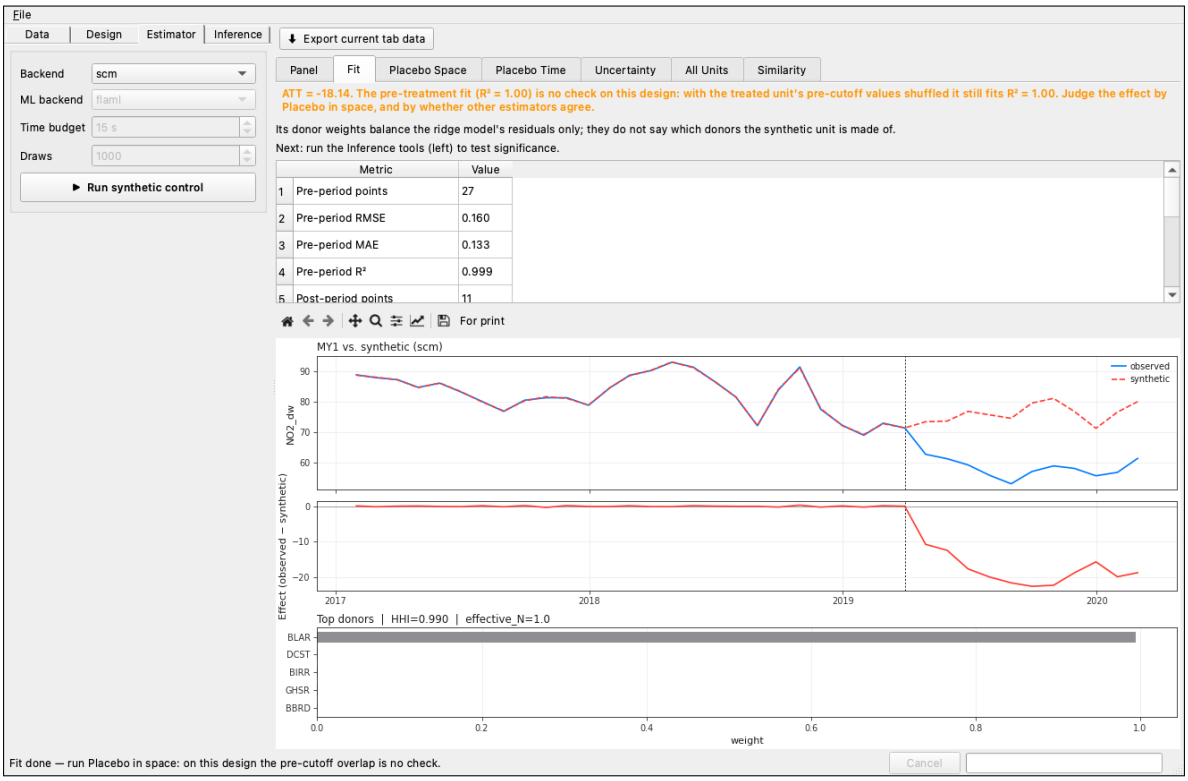



Figure 11.2: The augmented SCM (`scm`) fit for MY1: ATT $= -18.14\ \mu\mathrm{g\,m^{-3}}$ over the 11 post-cutoff months. Its pre-cutoff $R^2$ is 0.999, but with MY1's pre-cutoff values shuffled it still fits 0.9998 (both shown as 1.00), so the verdict does not read the fit and points to the placebo tests.

With 53 donors for 27 pre-cutoff months, this design is one where augmented SCM fits anything. Its top donor carries 99.5 % of the weight (HHI = 0.99), but those weights only balance the ridge model's residuals.

The estimators do not agree on this design. Run on the same panel, cutoff and donors:

| Backend | pre-cutoff $R^2$ | shuffled $R^2$ | pre-cutoff RMSE | ATT ($\mu\mathrm{g\,m^{-3}}$) |
|---|---|---|---|---|
| `scm` | 0.999 | 1.000 | 0.16 | −18.14 |
| `abadie` | −2.94 | −3.33 | 13.20 | −6.83 |
| `robust` | 0.37 | 0.16 | 5.27 | −9.48 |
| `did` | −0.23 | −0.09 | 7.36 | −20.67 |

The shuffled column is the baseline the **Fit** tab computes: only for `scm` does a series unrelated to the donors fit as well as MY1, so only its pre-cutoff $R^2$ goes unread. Classic SCM cannot fit MY1 at all: its weights are non-negative and sum to one, so the synthetic unit cannot exceed the highest donor, and MY1 is above every donor. Robust SCM fits it only loosely. The four estimates agree on the sign and differ threefold in size. That is a finding about the design, not a fault of the software: a treated unit outside its donors' range is the case synthetic control handles worst, and an analysis of it should report the spread, not one number.

## 11.4 Inference

**Run placebo in space** The **Placebo Space** tab draws every donor's placebo effect in grey and the treated unit's in red, then the treated effect inside a 95 % band of the placebos.

Green at $p \leq 0.05$, amber with "borderline" up to 0.15, amber otherwise. It also reports Abadie's RMSPE-ratio test: the treated unit's post/pre RMSPE ratio, its rank among the placebos and its p-value.

**Run placebo in time** Fake cutoffs every *Placebo every* steps (7), each with at least *Min pre-period* steps (30) before it. A fake post-period must fit before the true cutoff, so this needs a long pre-period; the tab says so when none fits. Green at $p \leq 0.05$; when $p$ is the smallest the number of fake cutoffs allows, the verdict says so and suggests a smaller step.

**Run pre-trend + event study** Was the fit parallel before the intervention? A slope test on the pre-cutoff residuals — its slope, p-value and an RMPE drift ratio, the fit's own parallel-trends check, which the **Fit** tab reports too — and the effect path on the event-time axis: the pre-period points should scatter about zero and the post-period should leave them.

**Run conformal interval** A finite-sample 95 % interval for the average post-cutoff effect, built from a sub-sampling null distribution of the effect's block means (Chernozhukov, Wüthrich & Zhu 2021); the number of sub-samples is the *Conformal draws* control. Green at $p \leq 0.05$.

**Run leave-one-donor-out** The control is refit leaving out each donor in turn: how far the remaining weights and the post-cutoff effect move when one donor drops. A result carried by a single donor stands out here.

**Uncertainty bands** Jackknife (leave each donor out) or bootstrap ($B$ refits, 100 by default). The verdict gives the mean post-cutoff effect and on what share of post-cutoff dates the band excludes zero; green at half or more.

**Run all units** Fits every unit as if it had been treated, with all the others as donors, and ranks them by the size of their post-cutoff effect. The treated unit should stand out: green when it is in the top tenth.

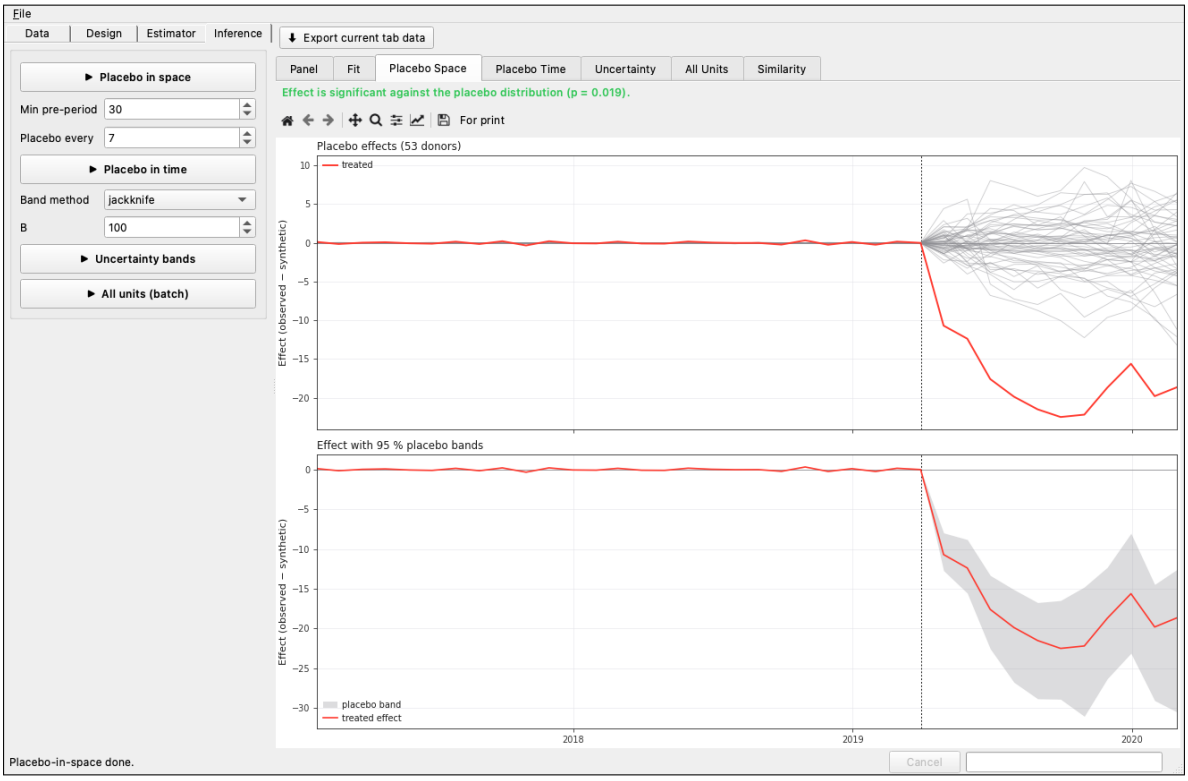



Figure 11.3: Placebo in space for the ULEZ: no donor's placebo effect comes near MY1's, so $p = 1/54 = 0.019$, the smallest 53 placebos allow. Every placebo is fitted by the same estimator, so this comparison is fair even where the pre-period fit is not informative.

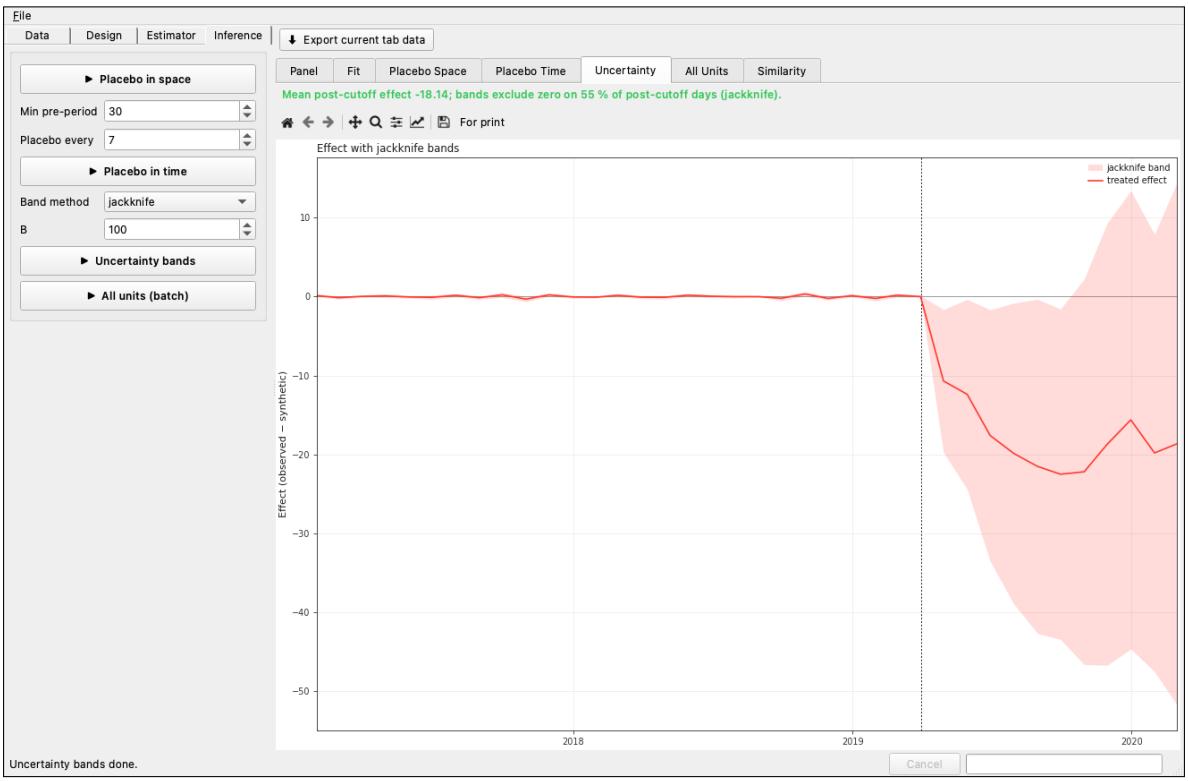



Figure 11.4: Jackknife bands: the effect is recomputed 53 times, leaving out one donor each time. The mean post-cutoff effect is $-18.14\ \mu\mathrm{g\,m^{-3}}$, and the band excludes zero on 55 % of the post-cutoff dates (the verdict says "days"; the panel is monthly).

# Chapter 12

# Getting data in

## 12.1 Input format

*Open Data…* (Ctrl+O) reads CSV and other delimited text (`.csv`, `.tsv`, `.txt`, `.dat`; comma, semicolon, tab, bar or space, a decimal comma, UTF-8 or a Windows code page — each told from the file), any of them gzipped, Excel (`.xlsx`, `.xlsm`, `.xls`), Parquet, Feather and JSON. The table needs:

- its times: a column named `date`, `datetime`, `time`, `timestamp` (any case, or their Chinese words), a date column with a time-of-day column beside it, or `year`, `month`, `day` (and `hour`) columns;
- the pollutant(s), numeric;
- the meteorology and any other predictors, numeric.

The times may be written in any of the usual ways: ISO 8601, `31/01/2020 13:00` (day first) or `01/31/2020` (month first) — the order told from the values, and said under the file's name when they do not tell —, `31.01.2020`, Chinese year–month–day writing, `2020013113`, Unix seconds or Excel's serial days. How they were read is shown under the file's name. One row per timestamp, hourly or coarser. A table of several sites in one file belongs in the SCM Studio, or in a script with `do_all_multisite`. A table without times is refused, with a pointer to the SCM Studio for panel data. Missing values are allowed: rows without a target are dropped, missing predictors filled (Section 3.1).

## 12.2 The clock: time zone

The *Time zone* under the file's name is the clock the analysis keeps and its figures and tables show: the site's local time, or UTC (the default). Times written without an offset are taken to be on it, as a monitoring network's local-time record is; times written with one (`+08:00`, `Z`) are put on it. The hour and the weekday the model learns from are then the site's own — a rush hour stays at its hour through summer time. Meteorology and trajectories fetched on UTC (the Data and Transport Studios) are put on this clock as they are joined, so an hour of ERA5 meets the hour of the measurement it belongs to. The choice is kept for the next session; changing it reads the open file again. In a script, `nm.io.read_time_series(path, zone=...)` reads a table the same way.

## 12.3 Data Studio: air quality networks and meteorology

*Get data* (Ctrl+D) assembles a model-ready table without leaving the app (Figure 12.1):

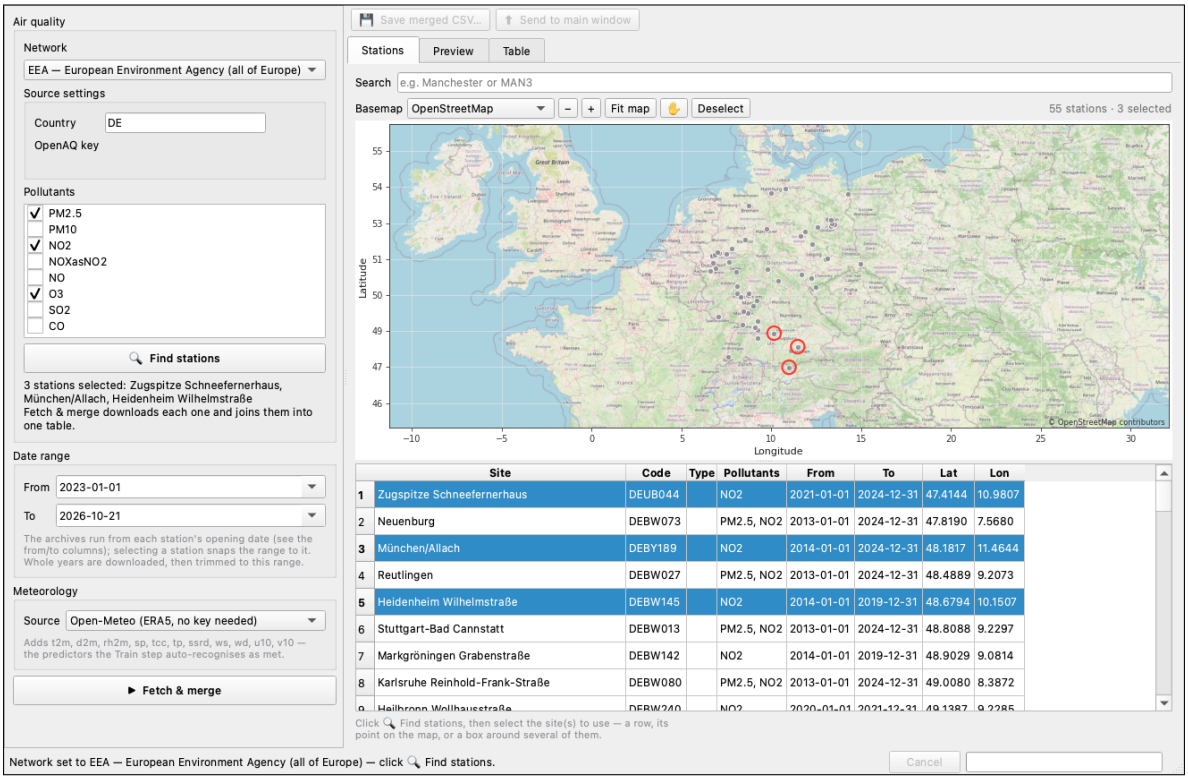



Figure 12.1: The Data Studio after a search on the EEA network for `DE`: the stations on the map, three picked and ringed, their rows highlighted in the table below — *Fetch & merge* would download all three into one table. Network, pollutants, date range and meteorology source on the left; the *Source settings* group belongs to the non-UK networks.

1. **Air quality.** Choose a network. The six UK networks are AURN (the national network, about 210 sites), AQE, SAQN, WAQN, NI and LMAM (locally managed), about 1500 stations in all; they need neither a country nor a key. The *EEA* row searches the European Environment Agency's reporting network for one country (ISO-2, e.g. `DE`): a few hundred to a thousand stations, also with no key. *OpenAQ* covers most of the world and wants a free API key — paste it into *Source settings*, or set `OPENAQ_API_KEY`. Tick the pollutants ($PM_{2.5}$, $NO_2$ and $O_3$ by default), and *Find stations*. The **Stations** tab draws the sites at their coordinates on a map, with the table beneath it: a click selects the station under the pointer, a box drawn with the mouse selects every station inside it, and Command/Control-click adds one. The table follows either way, and the search field narrows both (a name, or an official code such as `MAN3` for Manchester Piccadilly); hovering a point names the station. The wheel zooms about the pointer, the + and − buttons step it, the hand button (or the middle button) pans, *Fit map* brings the stations back and *Deselect* (or Esc) empties the selection; double-click a station to zoom to its streets, and right-click for the menu — select what the view holds, copy a station's coordinates. The row above the map counts what the search found and how much of it is picked. A basemap lies under the points — OpenStreetMap, Esri's light canvas or satellite imagery, picked on the row above the map, key-free and credited below it — fetched when the view is shown.
2. **Date range.** The UK archives are whole calendar years, from each station's opening; selecting a station snaps the range to its record. The EEA record is dated pollutant by

pollutant — its UK stations end in 2019, because the UK stopped reporting to the agency after Brexit — and the range snaps to the years the chosen station covers; an OpenAQ location carries its last reporting date.

3. **Meteorology.** Open-Meteo's ERA5 archive (no key), or the Copernicus CDS (needs `~/.cdsapirc`). Open-Meteo adds `t2m`, `d2m`, `rh2m`, `sp`, `tcc`, `tp`, `ssrd`, `ws`, `wd`, `u10` and `v10` — names the *Columns* box recognises as meteorology. They come on UTC and are put on the window's clock (Section 12.2) as they are joined.
4. **Fetch & merge** downloads the hourly measurements and the meteorology for the site's coordinates and joins them on the hour. With several stations selected it does that for each and stacks them into one table, which carries a `site` column — a panel the modelling steps can group. **Preview** and **Table** show the result; *Save merged CSV...* writes it and *Send to main window* opens it there.

The UK archives are the openair `.RData` files R's `importUKAQ()` reads, parsed without R, so the data match an openair workflow. The EEA path reads the agency's own station map and downloads one public Parquet file per station and pollutant — a whole record in one download. OpenAQ answers per sensor; the $\mu g\,m^{-3}$ sensor is preferred where a location registers several, so its values share the UK and EEA scale (a ppm-only location keeps its own unit, in the `unit` of its rows), and it carries the six common pollutants: a ticked NO or $NO_x$ is skipped for it, with a line in the log. The Data Studio needs an internet connection; a table sent from it is held in memory, and the command log says so rather than writing a `read_csv` it could not repeat.

## 12.4 Transport Studio: back-trajectory predictors

Local meteorology does not say where the air came from. The Transport Studio turns HYSPLIT back-trajectories into predictors that do, and joins them onto the table (Figure 12.2). Its panel has two pages: *Trajectories*, for files already made, and *HYSPLIT*, for running them.

**Trajectory files.** *Add files...* takes existing HYSPLIT `tdump` files (no HYSPLIT installation is needed to read them). *Site name* tags the result with a `site` column, so batches for several receptors, built one at a time, each fill only their own site's rows. *Null trajectories shorter than* (72 h) blanks trajectories that stopped early because the meteorology ran out or the air left the domain, which would otherwise pass for short-range transport.

**Source regions.** Optional regions, as bounding boxes (name, lon/lat min and max) or polygons from GeoJSON or Shapefile files (`geo` extra). Each adds a residence-time feature: the fraction of the trajectory spent inside. Overlapping regions each count an endpoint inside both.

**Features.** Every trajectory becomes one row of: `traj_n_endpoints` and `traj_age_max_h` (quality: how far back it reached), `traj_dist_km` (straight-line reach), `traj_pathlen_km`, `traj_speed_kmh`, `traj_inflow_deg` (bearing from the receptor to the origin), `traj_height_mean` and `traj_height_min`, one `traj_resid_`*region* per region, and — where the `tdump` carries them — along-path rainfall, mixing depth, humidity, pressure and temperature.

**HYSPLIT (advanced).** With a local `hyts_std`, the studio can run the trajectories itself, on its *HYSPLIT* page, for a table of receptors, at a chosen height, hours back and time step, with GDAS1 meteorology it downloads or ARL files already on disk. The features are built with the source regions set on the *Trajectories* page.

**Joining.** *Send to main window* joins the features onto the open table: each row takes the latest trajectory that has *already arrived*, within one release interval — a nearest-time join would give 04:00 the 06:00 trajectory, air that has not yet arrived — and the new `traj_*` columns are ticked as predictors. The trajectories' times, UTC as HYSPLIT writes them, are put on the window's clock first (Section 12.2); the studio is given the table's times in UTC to start them from.

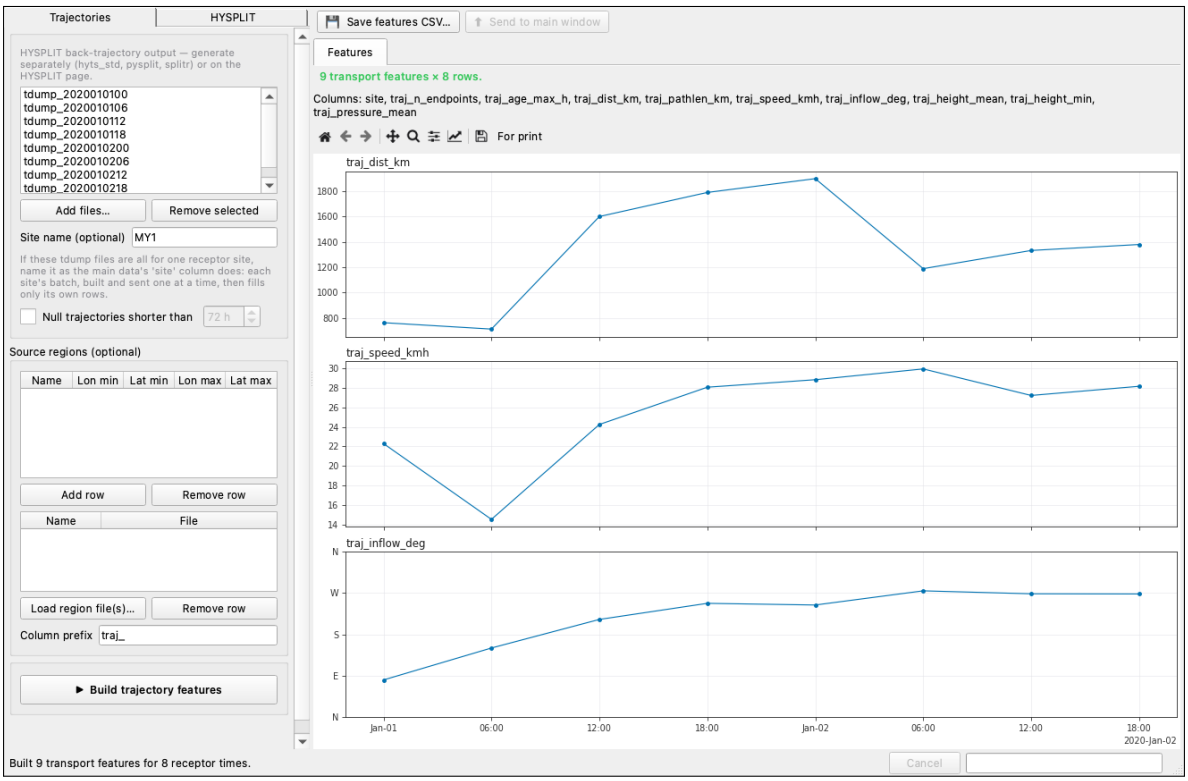



Figure 12.2: The two days of `tdump` files that ship with normet, built for receptor MY1: 9 features for 8 receptor times, released every six hours. Inflow turns from the east (80°) to the west (270°) over 1–2 January 2020.

**Caution.** Trajectories released every six hours are constant over six hourly rows. Under a random split a tree model uses them as a time stamp and its test $R^2$ is inflated (Section 3.1.1). Judge a transport-aware model with `month_ts`. To split $PM_{2.5}$ into local-weather and transport contributions, give the meteorological decomposition two groups, `local` and `transport` (Section 3.5).

## 12.5 Polar plot: where a pollutant comes from

**Analysis → Polar Plot...** draws a variable of the open table by wind direction and speed, as openair's `polarPlot` does (Figure 12.3): its mean, median, 95th percentile or maximum in bins of 10° of direction and of speed, on a compass, north at the top. The wind columns are found by name (`ws` and `wd`), else made from `u10` and `v10`, and can be chosen. In a script: `nm.polar_plot(df, target="NO2", ws_col="ws", wd_col="wd")`.

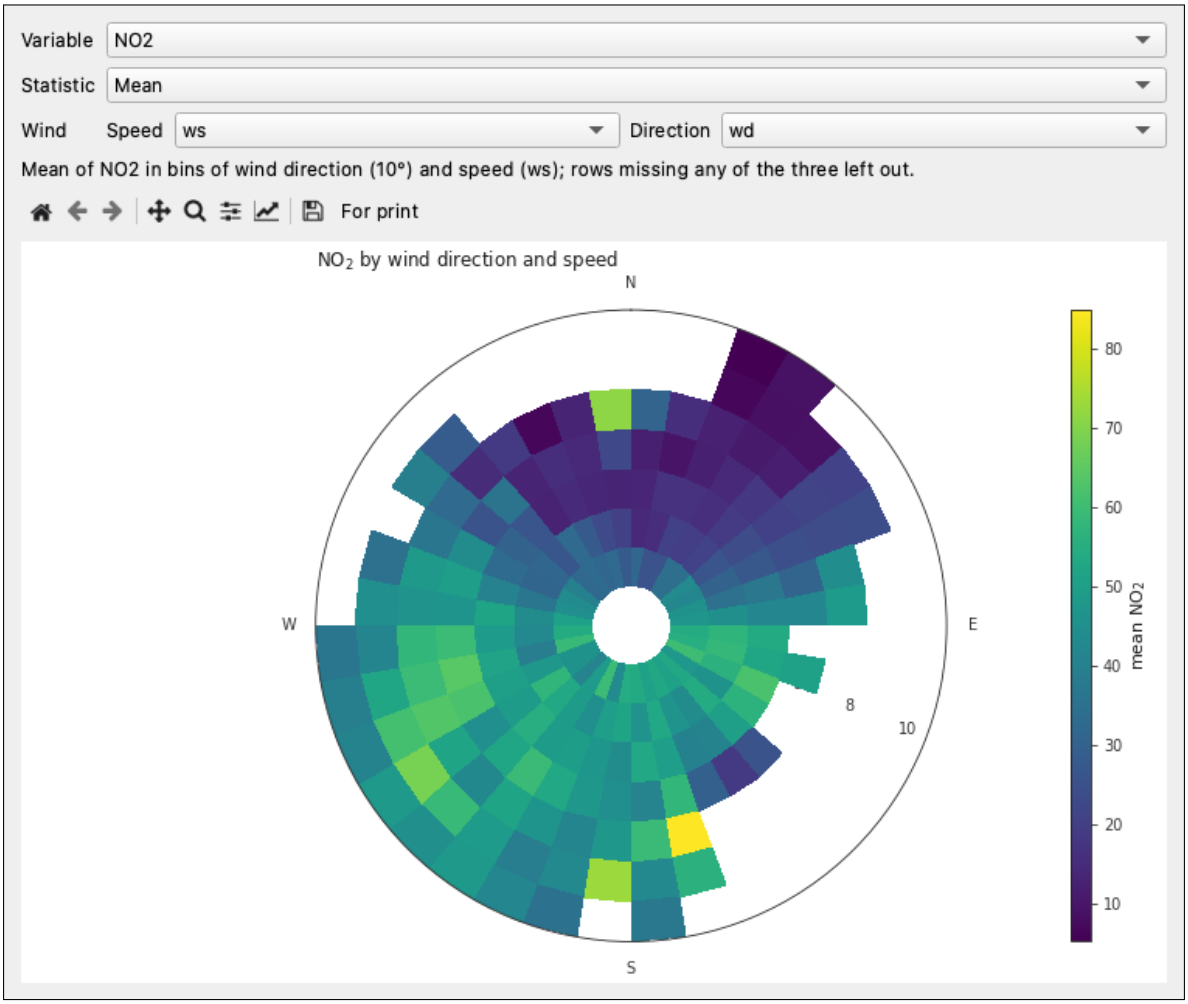



Figure 12.3: Mean $NO_2$ at MY1 by wind direction and speed: highest with winds from the south and south-west, lowest with winds from the north and north-east.

# Chapter 13

# Exporting and reproducing

## 13.1 Tables and reports

**Export result CSV** (Ctrl+E) The table behind the current tab: the loaded data on **Data**, the statistics on **Model**, the normalised series with its quantiles, the decomposition, every rolling window, the multi-scale series, the partial-dependence values.

**HTML report** (File → Export HTML Report, or the button; pick the report's language from its menu.) The current tab's result as one self-contained HTML file — figures embedded, no scripts or external links — with a summary of the model and the result, and its provenance: normet's version, the time, the platform and Python version, the host and user names, the shape and hash of the data, the panel's settings and their hash, and the seed. Remove the host and user names before sharing a report if they should not travel with it.

## 13.2 The command log

Every step that decides a result — opening a table, training, normalising, decomposing, a rolling run, a multi-scale run, partial dependence — is written to **View → Command Log...** as the normet call that does the same, with the arguments the window actually used. An entry is written only once its task has succeeded, from the call that was made, not from what the panel shows now. *Copy* puts the log on the clipboard, *Save as Script...* (also **File → Save Session as Script...**) writes it as a Python file that repeats the session, and *Clear* empties it.

The session behind this manual's figures saves as (paths shortened, imports and the multi-scale and partial-dependence calls left out):

```
# 21:09:36 Load example: London Marylebone Road NO2 (2020)
df = nm.datasets.load_my1()

# 21:10:09 Train model
df_prep, model = build_model(df, 'NO2', backend='flaml',
                             covariates=['ws', 'wd', 'temp', 'RH', 'atmos_pres', 'blh',
                                 'tcc', 'tp', 'ssrd', 'date_unix', 'day_julian', '
                                 weekday', 'hour'],
                             split_method='random', train_fraction=0.75,
                             model_config={'time_budget': 30, 'estimator_list': ['lgbm'
                                 ]},
                             seed=7654321)

# 21:10:11 Normalise
normalised = normalise(df_prep, model,
                       covariates=[...the same 13...],
```

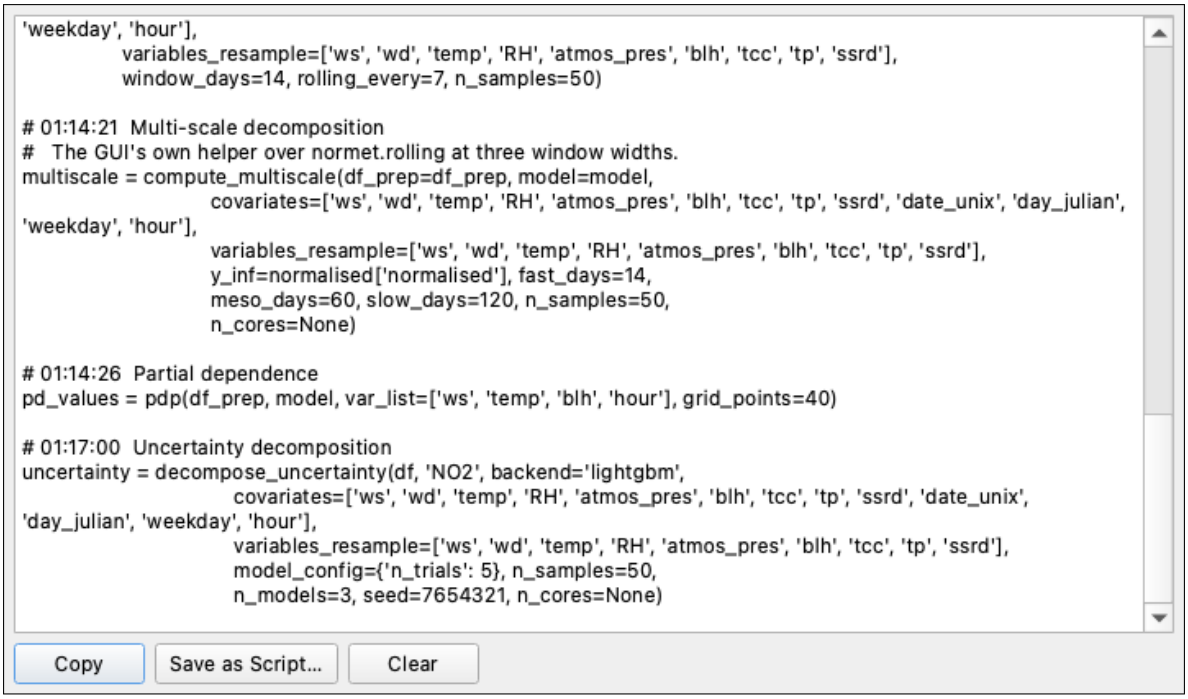



Figure 13.1: The Command Log after this manual's session, its last entries in view: the multi-scale run and the partial dependence, as the calls that repeat them.

```
                      variables_resample=['ws', 'wd', 'temp', 'RH', 'atmos_pres', 'blh
                          ', 'tcc', 'tp', 'ssrd'],
                      n_samples=300, n_cores=None,
                      return_quantiles=(0.05, 0.25, 0.75, 0.95))

# 21:10:22 Decompose
decomposition = decompose(df_prep, model, method='meteorology',
                          covariates=[...the same 13...], n_samples=100,
                          groups={'Wind': ['ws', 'wd'],
                                  'Temperature and humidity': ['temp', 'RH'],
                                  'Boundary layer and pressure': ['atmos_pres', 'blh'],
                                  'Sky and rain': ['tcc', 'tp', 'ssrd']})

# 21:10:25 Rolling normalisation
rolled = rolling(df_prep, model, covariates=[...],
                 variables_resample=[...the nine...],
                 window_days=14, rolling_every=7, n_samples=50)
```

Run again, it gives the same normalisation and decompositions from the same model; the model itself can differ slightly, since FLAML's search is bounded by wall-clock time (Section 3.2).

## 13.3 Figures for publication

A figure on screen is drawn at the tab's size; shrunk into a journal's 89 mm column its lettering shrinks with it. *For print…*, on every figure's toolbar, lays the figure out again at the size the journal prints it, each text at the size its guide gives that kind of text — a title, an axis label, a tick label, a legend — with the margins and the ticks worked out anew for that size, not scaled to it (Figure 13.2):

**Journal** Elsevier, Nature, Copernicus (ACP, AMT, …) or ACS (ES&T), each with what its own artwork guide asks, and a link to that guide.

**Width** The journal's columns (Nature 89 or 183 mm; Elsevier 90, 140 or 190 mm; ACS 3.33 or 7 in; Copernicus at least 80 mm), or a width of your own. A figure of one column of panels starts at the narrowest, panels side by side at the widest.

**Height** As the figure needs — its own proportion, raised until every panel has at least 12 mm for its data, within the journal's greatest height, and lowered to what is drawn where panels of a shape of their own (roses side by side) would leave the rest blank — or set.

**Panel labels** None, a b c, (a) (b) (c), A B C or (A) (B) (C), bold or not, from any letter (a figure that is panels c and d of a composite starts at c); each column of panels has its letters in a line.

**Line width, Marker size** The data lines' stroke; a time series takes round markers when the size is raised. Reference lines keep theirs.

**Legend** Where it sits (or drag it), its text size, its columns, its frame. A legend laid out in a row wider than the printed figure is given fewer columns.

**Formulas** Written as chemistry writes them: $NO_2$, $PM_{2.5}$, $SO_4^{2-}$.

**Titles** The figure's and the panels' titles are left to the caption, as journals ask; ticked, they are drawn.

**Raster resolution** 300, 600 or 1200 dpi, for TIFF and PNG.

Under the settings, the checks read the figure as it is laid out against the journal's guide: its width and height, its smallest text and its smallest sub- or superscript (Nature asks for 5 pt at least and 7 pt at most, Elsevier 7 pt, ACS 8 pt), the face it is drawn in (Arial or Helvetica where the guide names them), tick labels that run into one another, and how many panels it has (ACS: four at most); a check the figure fails is shown in red, and all are read again as the settings change. The preview fits the window, or shows the figure at its actual printed size. On the preview, double-click a title, an axis label or a panel letter to edit its text and size, a tick's numbers to size and turn them (and give that tick a text of its own), the legend to place it, or a colour bar to put it on another side of its panel and give its width and height on the page in millimetres; drag the legend or a colour bar where it reads best. *Save...* writes the formats the journal takes — PDF, EPS or SVG with the text kept as editable text and the fonts embedded, TIFF or PNG at the resolution chosen — at exactly the size shown, on a white page. While the dialog is open the figure is in the report's language (the one last picked under File → Export HTML Report). When it closes the figure goes back to its tab as it was; the journal, width, panel labels, formulas and resolution are kept for the next figure. Check the journal's current guide before submitting.

## 13.4 Saving and reopening a session

**File → Save Results...** (Ctrl+S) writes the session to one file (`.normet`): the table, the panel's settings, the model, every step's result and the command log. **Load Results...** reads one back into the window as it was — each result is handed to its step's own handler, so its tab, verdict and figure are drawn again — and a notice says what was restored. After each step that finishes, the window also keeps the session as `~/.normet/last_session.normet`; **Reopen Last Session** reads it, should the window have been closed, or have crashed, before the results were saved. A Chronos-2 checkpoint is not written (it is the pretrained model, about 500 MB): load it again to go on. A session file is a Python pickle: open only one you or the window wrote.

**File → Load Session Script...** runs a script the command log wrote (Section 13.2) and brings its results into the window the same way: its table, model and results, and the panel set from the calls it made. The script is Python and runs as it is written, so the window asks first.

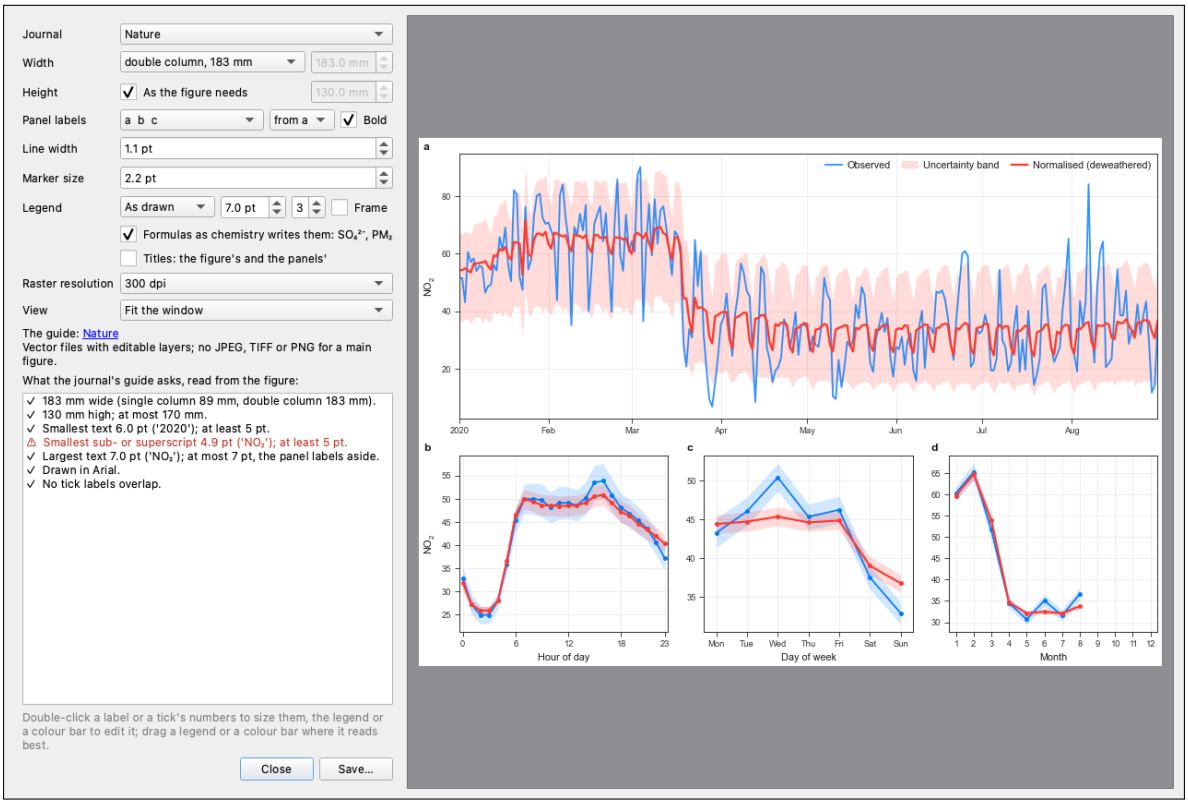



Figure 13.2: The normalised series laid out for a Nature double column: 183 mm wide, its panels lettered, beside what Nature's guide asks of it — a subscript at 4.9 pt, below the 5 pt Nature asks, is the one check it fails: Nature's 7 pt ceiling leaves no room for more.

## 13.5 A Methods paragraph

**File → Methods Paragraph...** writes what the session ran as a paper's Methods says it, ready to copy (Figure 13.3): the record (its length, span and step), the model (backend, search, predictors, split and seed) and its held-out fit, then each step run — the normalisation's draws and resampled variables, the decomposition, the rolling windows, the multi-scale widths, the partial-dependence variables — with the references the methods rest on. It is written from each step's call as it ran, not from the panel as it stands, and in the window's language. Check it against what you did.

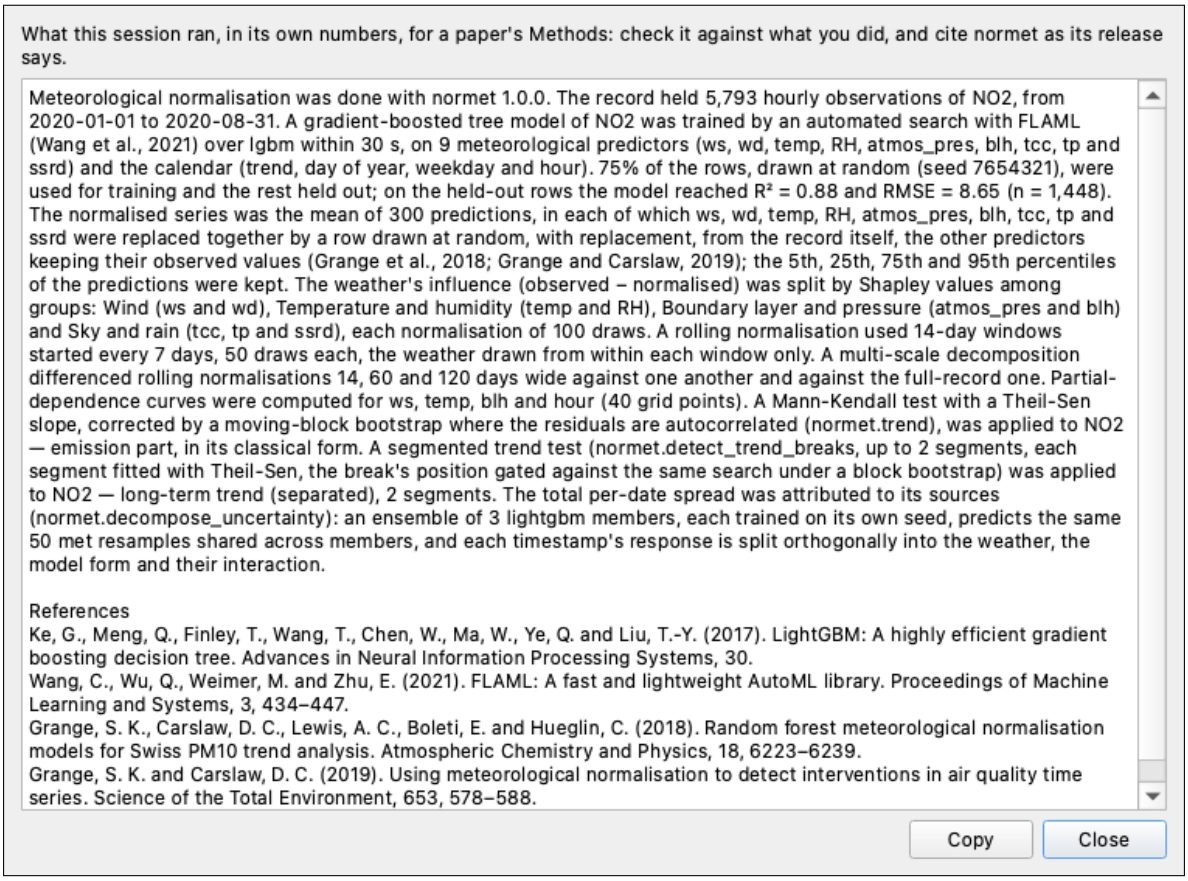



Figure 13.3: The Methods paragraph of this manual's session: the record, the FLAML model and its held-out fit, and each step that was run, with its references.

## 13.6 Configurations and history

**File → Save Config...** writes every setting of the panel to JSON — target, predictors, backend, split, budget, estimators, seed, and every step's settings — and **Load Config...** sets the panel from one; the data are not part of it. **File → Run History...** lists every run with its time, task, target, backend, rows and a detail (the held-out $R^2$ for a training run, the samples for a normalisation).

# Chapter 14

# The Assistant

The Assistant is a language model working in the window. It is open on the right when normet starts; **View** → **Assistant** (Ctrl+Shift+A) closes it and opens it again. It reads what the window holds — the table, the model's held-out fit, the normalised series, the decomposition — as normet's own readings of them, in text, and runs the window's steps the way their buttons do, so what it does appears on screen as your own runs do and goes into the command log.

The conversation reads as a chat does: your messages on the right, the replies as their Markdown reads (code shaded, tables ruled), each tool the Assistant calls one quiet line — its dot grey while it runs, green when it answered, red when it was refused, the refusal under it — whose result opens with a click. Ctrl/Cmd + and − (or the wheel with the key held) size its text, and Ctrl/Cmd 0 resets it; the size is kept. You write in the rounded box at its foot, which grows with what you write: Enter sends, Shift+Enter starts a new line. *Send*, *Stop* and *New chat*, which starts the conversation again, are inside the box.

Where it helps most is what to model and how to read the result. It *proposes* a target and predictors as a card with an *Apply* button, saying what it left out and why; nothing in the panel changes until you click Apply, and it trains or normalises only once you have agreed. It is told to quote only numbers the tools return.

## 14.1 Full automatic

*Auto*, beside *Key...*, lets the Assistant work without waiting for you. With it on, the target and predictors it proposes are applied at once (the card says *Applied*), and it works the analysis through in the window's order — the data, the set-up, training, normalisation, decomposition — reading each result as it comes. A model that stops with steps left is told what is left and goes on, up to eight times; two reminders in a row that change nothing end the turn, as a model that cannot do a step should not circle. A call to the model that fails, or an empty reply, is made again, twice at most. Each reminder and retry is a line in the conversation. *Stop* ends it at any time; the choice is kept for the next session, and *What full automatic does...* under the button's arrow says all this. A command-line agent has the set-up applied at once too, and works the chain as far as its own loop allows.

## 14.2 Choosing a model

**An API model** named `provider:model`: `anthropic`, `openai`, `deepseek`, `gemini`, or a local `ollama` or `lmstudio` server — for example `anthropic:claude-sonnet-5-5`, `deepseek:deepseek-chat`, `ollama:qwen3:8b`. Its key is read from the macOS keychain (*Key...* stores it there) or the provider's environment variable (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `DEEPSEEK_API_KEY`, `GEMINI_API_KEY`), and written nowhere else. Local servers need no key.

**A command-line agent** signed in to your own plan: Claude Code, Codex, Google Antigravity's `agy` or GitHub Copilot, with the effort it thinks at. The window serves its tools to the agent over MCP on 127.0.0.1 with a token drawn for that window, and gives it no shell and no files of its own.

What the Assistant sends to a remote model is the readings — summaries, statistics, captions — and your messages, not the table.

# Chapter 15

# Scripting: API, CLI and MCP

## 15.1 Python API

Everything the window does is a function of the `normet` package. The one-call pipeline prepares, trains and normalises:

```
import normet as nm

df = nm.datasets.load_my1()
met = ["ws", "wd", "temp", "RH", "atmos_pres", "blh", "tcc", "tp", "ssrd"]
predictors = met + ["date_unix", "day_julian", "weekday", "hour"]

out, model, df_prep = nm.do_all(
    df=df, target="NO2", backend="flaml",
    covariates=predictors, variables_resample=met,
    n_samples=300, n_cores=4,
)
print(nm.modStats(df_prep, model)) # training, testing and all rows
```

Step by step:

```
df_prep, model = nm.build_model(df, "NO2", backend="flaml", covariates=predictors,
                                model_config={"time_budget": 120})
norm = nm.normalise(df_prep, model, covariates=predictors,
                    variables_resample=met, n_samples=300,
                    return_quantiles=[0.025, 0.975])
auto = nm.normalise_auto(df_prep, model, covariates=predictors,
                         variables_resample=met) # stops when converged
emi = nm.decompose(df_prep, model, covariates=predictors, method="emission")
mets = nm.decompose(df_prep, model, covariates=predictors, method="meteorology",
                    groups={"wind": ["ws", "wd"], "rest": met[2:]})
roll = nm.rolling(df_prep, model, covariates=predictors, variables_resample=met)
curves = nm.pdp(df_prep, model, var_list=["ws", "temp"])
```

Further functions, all exported at the top level:

| Function | Does |
|---|---|
| `do_all_unc` | the pipeline over `n_models` seeds, with confidence bands |
| `do_all_multisite` | per-site pipelines in parallel |
| `cv_score` | walk-forward cross-validation |
| `run_scm`, `scm_all` | a synthetic control; every unit as if treated |
| `placebo_in_space`, `placebo_in_time` | placebo tests |
| `uncertainty_bands`, `effect_bands_space` | jackknife/bootstrap and placebo bands |
| `scm_diagnostics`, `conformal_effect_interval` | fit diagnostics; conformal interval |
| `bayesian_scm` | Bayesian SCM (`bayes` extra) |
| `detect_events`, `anomaly_scores` | events in a normalised series |
| `normalise_plot`, `decomposition_stack`, `scm_dashboard` | the window's figures |
| `make_run`, `save_run`, `generate_html_report` | provenance and reports |
| `trend`, `mann_kendall`, `theil_sen` | trend tests with the block-bootstrap correction (the **Trend** tab, Section 8.3) |
| `detect_trend_breaks` | segmented trend: where the level or slope breaks (BIC + block-bootstrap gate) |
| `decompose_uncertainty` | the total spread attributed to weather, model form and their interaction (the **Uncertainty** tab, Section 8.4) |
| `make_memory` | on-disk caching |
| `Chronos2Estimator` | zero-shot de-weathering (`foundation` extra) |

The configuration of each long function is also a dataclass (`NormaliseConfig`, `DecomposeConfig`, `RollingConfig`); keywords override its fields. The guides at https://normet.readthedocs.io cover each area with worked examples, and the notebooks in the repository's `notebooks/` folder reproduce the paper's case studies.

## 15.2 Command line

With the `cli` extra, `normet` runs the pipelines on a CSV or Parquet file:

```
normet do-all my1.csv --target NO2 \
    --covariates ws,wd,temp,RH,atmos_pres,blh,tcc,tp,ssrd,date_unix,day_julian,
        weekday,hour \
    --resample-vars ws,wd,temp,RH,atmos_pres,blh,tcc,tp,ssrd \
    --n-samples 300 --out normalised.csv --run-path my1_run
normet report my1_run.joblib --out my1_report.html
normet decompose my1.csv --target NO2 --covariates ... --method emission --out emi.
    csv
normet scm ulez_panel.csv --treated MY1 --cutoff 2019-04-01 --date-col date \
    --unit-col code --outcome-col NO2_dw --backend scm --out scm.csv
normet cv my1.csv --target NO2 --covariates ... --n-splits 5 --out cv.csv
normet deweather site.csv --target PM2.5 --met-vars t2m,blh,u10,v10 --out dw.csv
normet info # version and the backends installed
```

Every command takes `--config file.yaml`, whose keys are the options' names; an option on the command line overrides the file. `--run-path` also saves a `NormetRun` archive (`.joblib` with a `.meta.json` sidecar) that `normet report` turns into HTML or Markdown. The command line's `scm` offers `scm`, `mlscm`, `abadie`, `did` and `mcnnm`.

## 15.3 AI assistants over MCP

With the `mcp` extra, `normet-mcp` serves normet to an AI assistant over the Model Context Protocol:

```
claude mcp add normet -- normet-mcp # Claude Code
# Claude Desktop, claude_desktop_config.json:
# {"mcpServers": {"normet": {"command": "normet-mcp"}}}
```

The assistant can list the examples, inspect a table, start `do-all`, `decompose`, `scm` and `cv` jobs, follow and cancel them, read a run or a result, draw a figure and write a report. Each fit is a normet command run in its own process and folder (`$NORMET_MCP_HOME`, `~/.normet/mcp` by default), so its command line is its record and runs again without the assistant. The assistant is given normet's readings of a table or result, not the data; files are read only under `$NORMET_MCP_ROOTS` (your home folder by default). The server asks the assistant to propose the target, predictors and donor pool with its reasons, to let you confirm them, and to quote only the numbers the tools return.

# Appendix A

# Defaults

| Setting | Window | Library |
|---|---|---|
| Split method / training fraction | random / 0.75 | random / 0.75 |
| FLAML time budget | 60 s | 90 s |
| FLAML estimators | lgbm | lgbm |
| FLAML metric | $R^2$ | $R^2$ |
| LightGBM search trials | 20 | — |
| Seed | 7654321 | 7654321 |
| Normalisation samples | 300 (Chronos-2: 8) | 300 (Chronos-2: 8) |
| Resampled variables | recognised meteorology | every non-calendar predictor |
| Quantiles | 5, 25, 75, 95 % when ticked | none |
| Decomposition samples | 300 | 300 |
| Attribution | auto | sequential (Shapley with groups) |
| Rolling window / step / samples | 14 d / 7 d / 100 | 14 d / 7 d / 300 |
| Multi-scale widths | 14 / 90 / 365 d | — |
| PDP grid points / range | 50 / 1st–99th percentile | 50 / 1st–99th percentile |
| `normalise_auto` | — | batches of 100, P95 RSE $< 3\,\%$, 3 in a row, $\leq 5000$ |
| Chronos-2 context / horizon | 2048 / 168 rows | 2048 / 168 rows |
| SCM backend | scm | scm |
| Placebo in time: min pre / every | 30 / 7 steps | — |
| Uncertainty bands | jackknife, $B = 100$ | jackknife, $B = 200$, 80 % donors |

# Appendix B

# Where normet keeps its files

| What | Where |
|---|---|
| Recent tables (eight) | `~/.normet/gui_recent_files.json` |
| Run history | `~/.normet/gui_run_history.jsonl` |
| Window geometry, appearance, language, Assistant model | the system's settings store (`apai-sys/normet-gui`) |
| Font cache (installers) | `~/.normet/matplotlib` |
| The last session (after each step) | `~/.normet/last_session.normet` |
| Assistant API keys | macOS keychain, service `normet` |
| MCP jobs | `$NORMET_MCP_HOME`, default `~/.normet/mcp` |
| Chronos-2 weights | the Hugging Face cache |

Nothing else is written unless you save it. The print dialog's choices, the Assistant's text size and the report's language are kept in the settings store with the rest.

# Appendix C

# Troubleshooting

**"The CSV must contain a 'date' column"** Rename the timestamp column to `date`. Panel data (several units) belongs in the SCM Studio.

**The model's verdict is red** The predictors do not explain the target. Check that the meteorology is ticked and on the same time base as the target, lengthen the budget, or add estimators. Normalising with a poor model returns little more than the observed series.

**A meteorological column is not ticked** Only the names in Section 6.1 are recognised; tick others by hand, on the **Data** and **Normalise** pages.

**Exact Shapley values over $k$ features need $2^k$ normalisations** More than ten variables by Shapley: give groups, or use sequential attribution.

**"... not among the variables trained on"** A group (from a configuration read in) names a variable the model was not trained on; the message names it.

**"'rdata' is required but not installed"** *Find stations* reads the UK archives with `rdata`, part of the `data` extra: `pip install "normet[data]"`. The installers include it.

**Multi-scale asks for the normalisation first** It differences against the normalised series.

**A multi-scale band was skipped** Its width leaves fewer than two windows on the record; lower it.

**Rolling and PDP are disabled** They need an AutoML model; the Chronos-2 backend has no response surface to vary.

**Chronos-2 is greyed out** Install `normet[foundation]` (and `normet[finetune]` to fine-tune).

**Chronos-2's verdict is red** Shuffling the weather barely moves its forecast: it is autoregressing. Check the meteorological columns, or use an AutoML backend.

**SCM: "The pre-treatment fit is no check on this design"** A series shuffled out of any relation to the donors fits the pre-period as well as the treated unit does: there are too many donors for the pre-cutoff dates. Judge the effect by Placebo in space and by other estimators, or use fewer, better-chosen donors (the similarity ranking can help) or a longer pre-period.

**SCM: "Donor pool too small"** Tick at least two donors.

**SCM: no valid fake cutoffs** The post-period must fit before the true cutoff with *Min pre-period* steps before it; lower *Min pre-period* or *Placebo every*, or move the cutoff later.

**A task runs too long** *Cancel* frees the window at once; the computation finishes in the background and its result is discarded. Lower the samples, the budget, or the number of variables decomposed.

**A notice asks for a step first** It names what is missing and carries the step on a button; it goes by itself once the step has been taken.

**The window closed before the results were saved** **File → Reopen Last Session** brings back the session as it was after its last step.

**The parameter panel is missing** It is folded away: *Expand Panel* on the action bar, or Ctrl+B.

**A task failed** The dialog shows its last line; the full traceback is in the **Log** dock.

**macOS: "Normet cannot be opened"** The app is unsigned: right-click it and choose *Open* once (Section 2).